HELMHOLTZ AI | ARTIFICIAL INTELLIGENCE COOPERATION UNIT

Helmholtz-Zentrum hereon

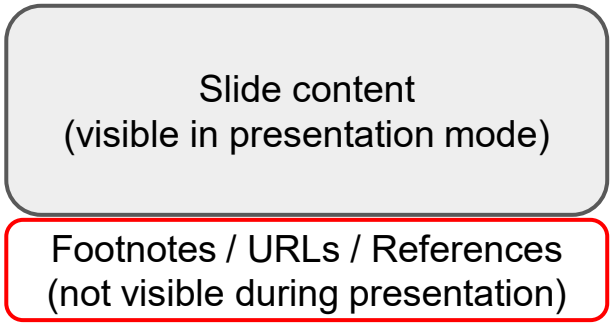
Understanding Transformers
Overview, Internals and Insights

October 2026

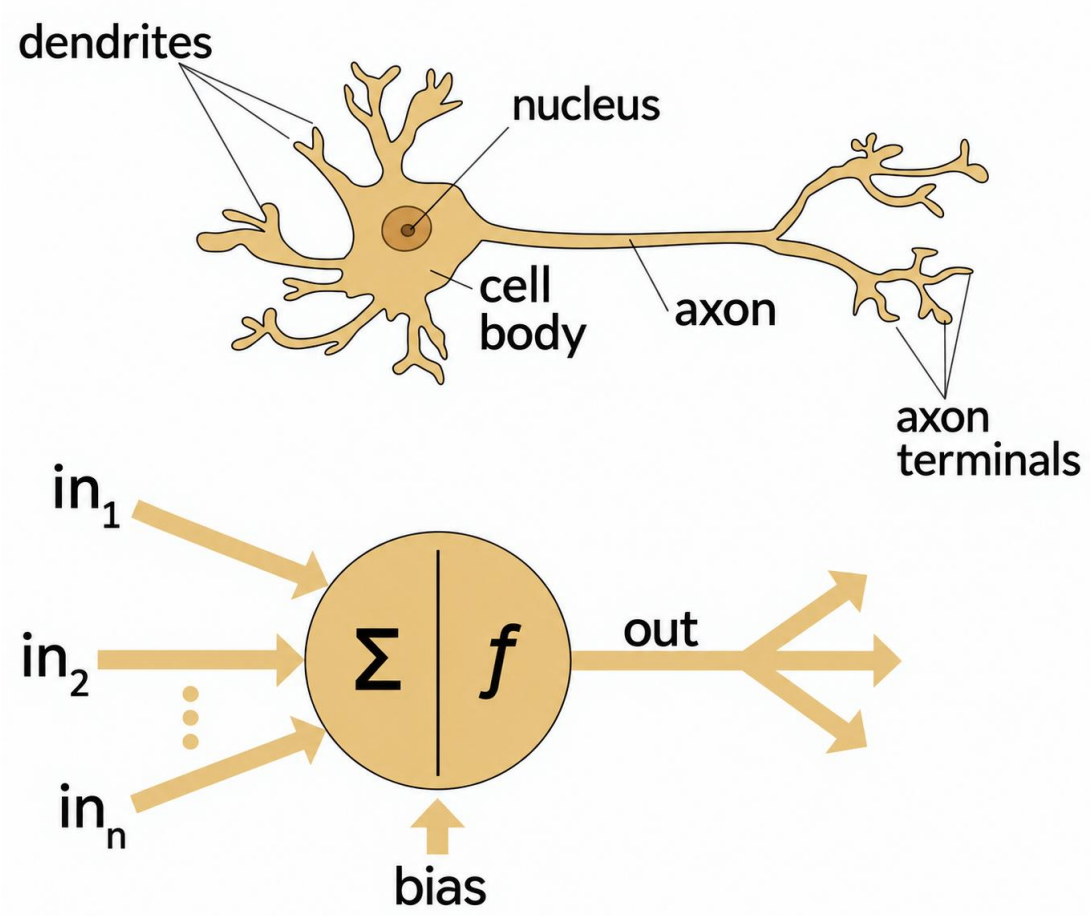
Danu Caus

About This Workshop

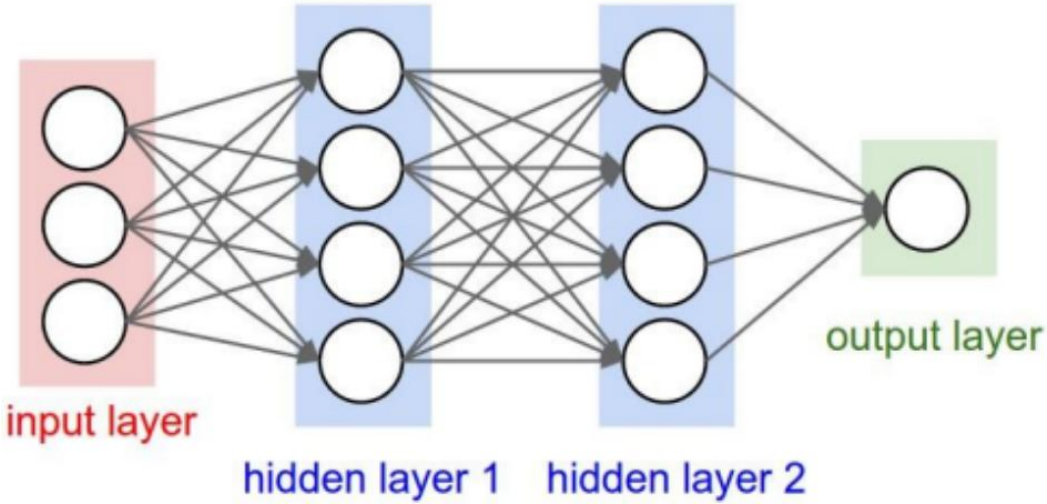
- This resource is intended to introduce neural networks, generative AI at a high abstract level and subsequently focus on **explaining Transformers** in more depth to a scientific audience.
- Transformers will be presented in the broader context of AI, building the stage from zero, step by step.
- We **will act** therefore **as a synthesizer** of many resources created by the broader AI community.
- Hence, many thanks! to all the creators of the helper material. **All credits** and references are visible on the **last slides**.
- Same references are also given in the **footnotes section** of each individual slide, which is **not visible in presentation mode**, but will be useful for referencing at home for extra study, if need be.



Artificial Neural Nets and Brain Parallels

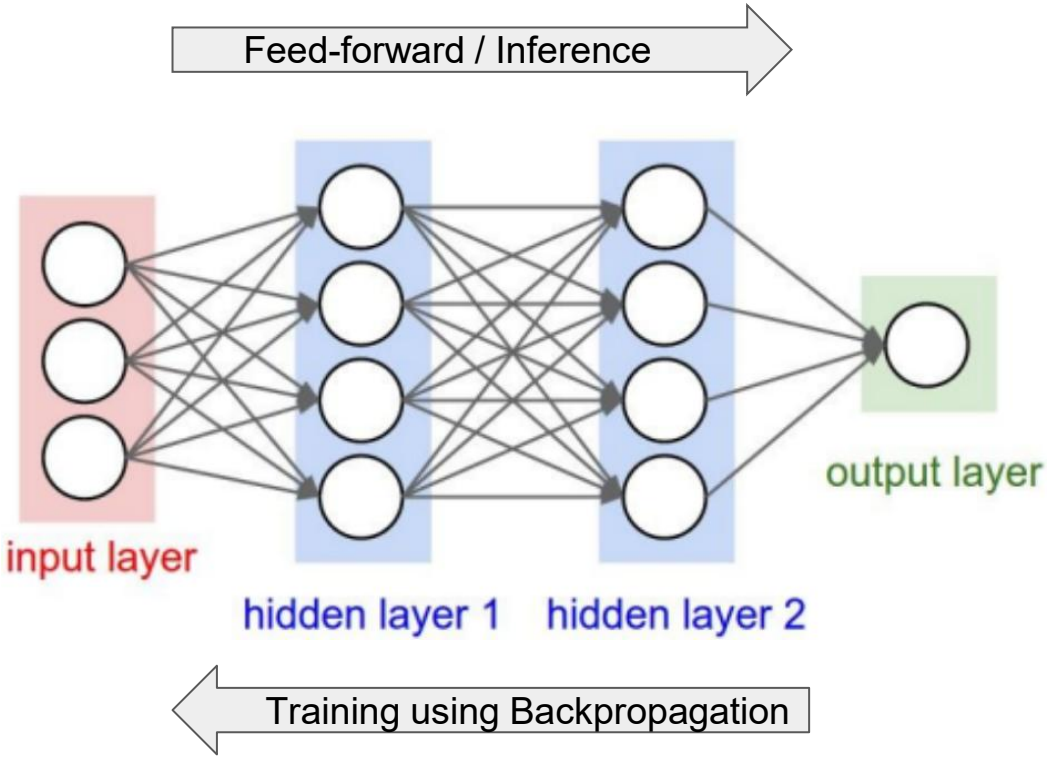


Artificial Neural Networks (ANNs)



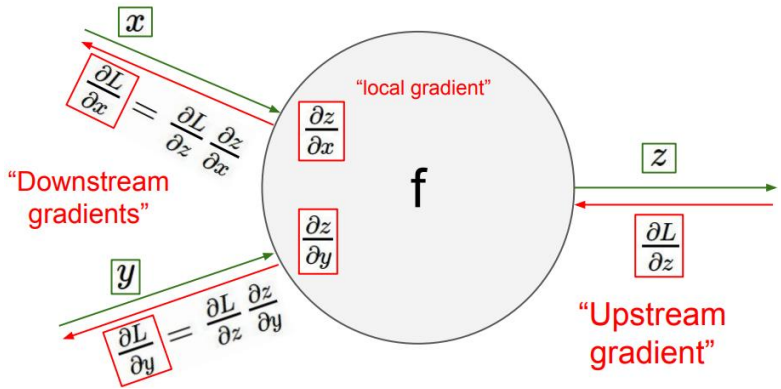
This is a **Multi-Layer Perceptron / MLP** and it is used as one of the building blocks of the Transformer.

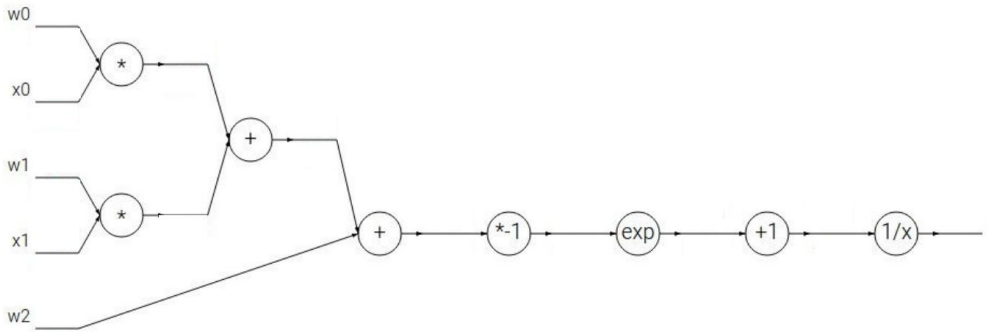
Backpropagation



CS231n course from Stanford University

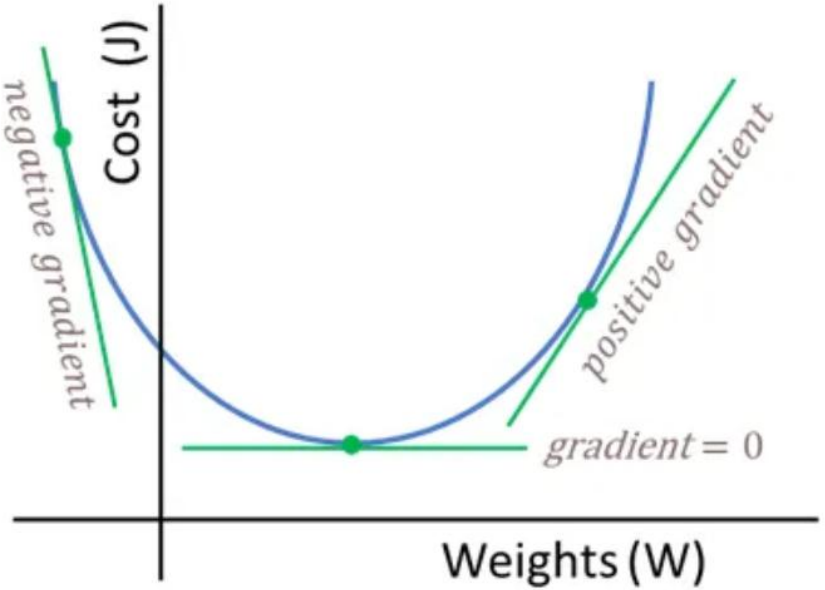
Backpropagation using Local Gradients and the Chain Rule





- The Chain Rule is implemented with the help of **local gradients**.
- We **recursively multiply** the local derivatives.
- Backpropagation is a **recursive** application of the chain rule, backwards through the **computational graph**.

Cost function



- Naming conventions in the literature: **Cost / Loss / Error** function

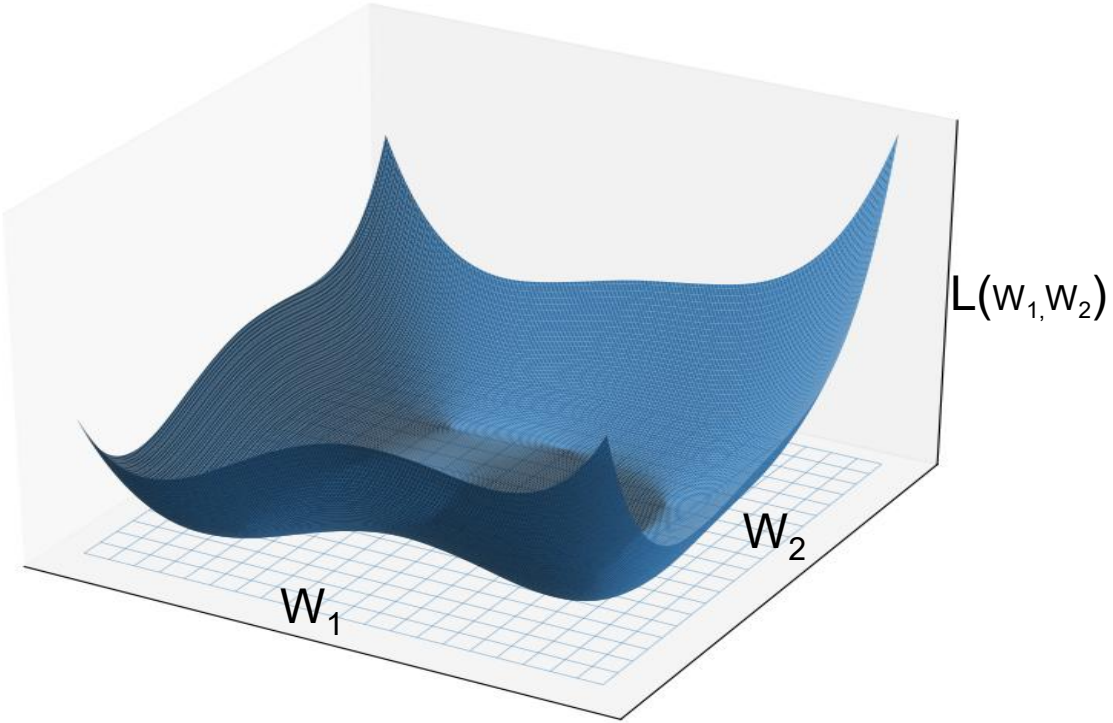
- The task is to find a **global minimum**, or **good** enough **suboptimal minima** points via the **gradient descent** optimization procedure, i.e. parameter search.

Updating Weights with Gradient Descent

$$W_{new} = W_{old} - \alpha \underbrace{\frac{dJ}{dW}}_{\text{gradient}}$$

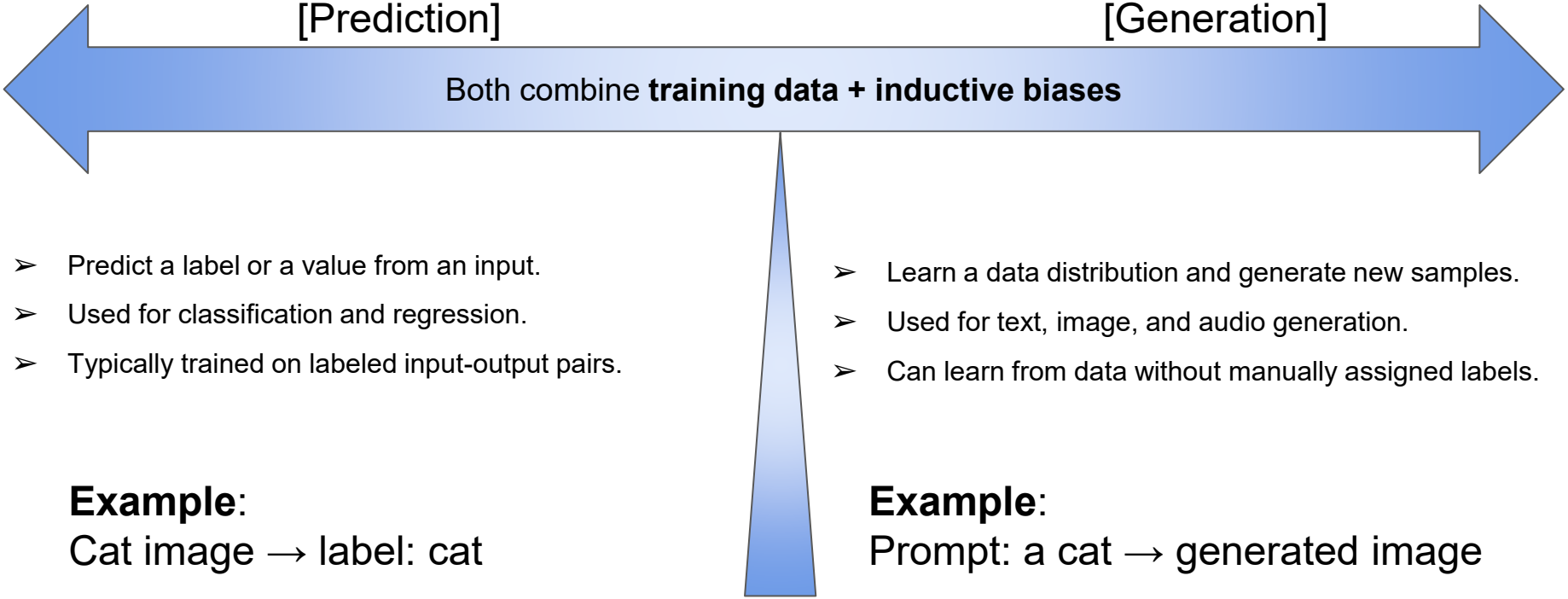
- **J** and **L** are usual notations for the **Loss** / **Error** / **Cost** function, i.e. the difference between what the model **predicts** and what it should predict according to the **ground truth**.
- The **weights are updated** in the direction of **negative** gradient, so that the **cost function is minimized** as much as possible.

Optimization in High-Dimensional Spaces



- The **cost function is** much more **complex** for a **typical neural net** => Millions and even billions of weights!
- Neural Network gradients have the ability to **navigate the many dimensions skillfully** though.
- And actually, having many dimensions **helps the gradient not to get stuck** in bad suboptimal points.

Discriminative and Generative Models



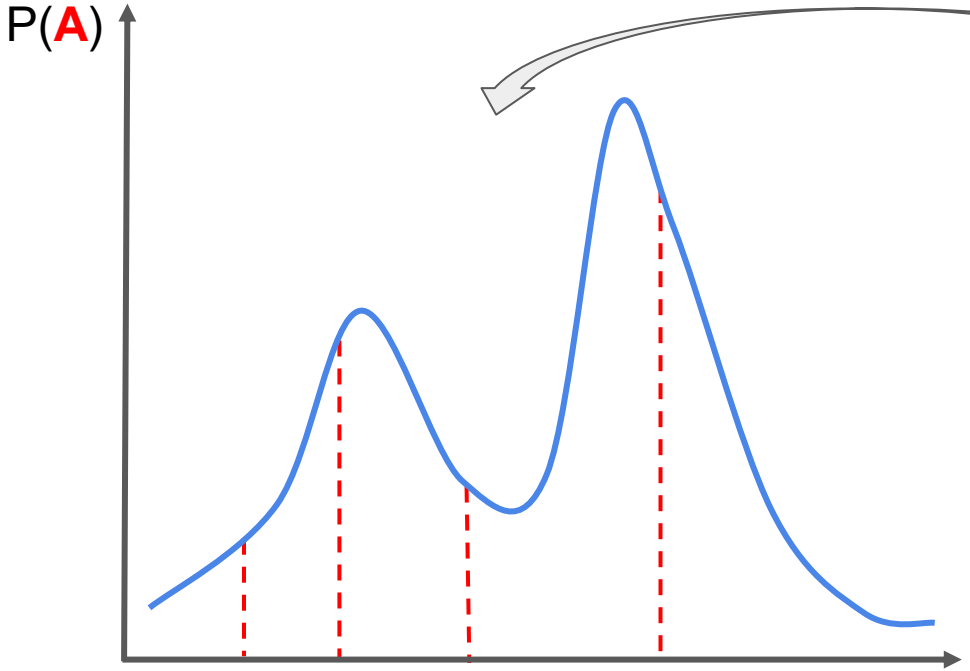
- Predict a label or a value from an input.
- Used for classification and regression.
- Typically trained on labeled input-output pairs.

Example:
Cat image → label: cat

- Learn a data distribution and generate new samples.
- Used for text, image, and audio generation.
- Can learn from data without manually assigned labels.

Example:
Prompt: a cat → generated image

Encode a “**world model**” in the form of a very-high-dimensional / complex **probability distribution.**



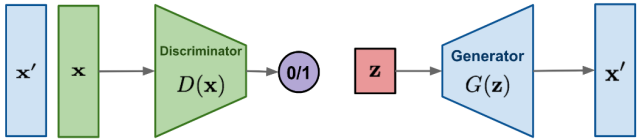


- Internet
- Books
- Images
- Videos
- Audio
- Robotic-Arm configurations
- Etc.

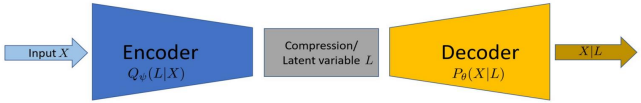
User prompts will statistically “sample **A**nswers” from the learned model.

Mainstream Generative AI models

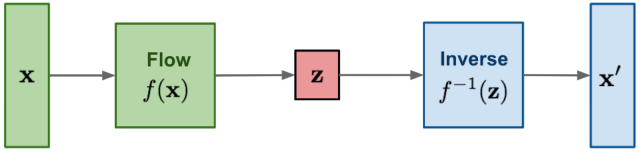
- Generative Adversarial Networks (GANs) → **Adversarial** training



- Variational Autoencoders (VAEs) → **Probabilistic** autoencoders

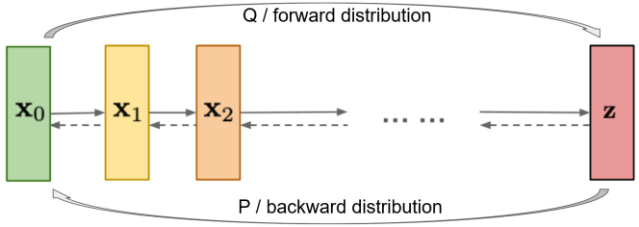


- Flow-based models → **Invertible mapping** between distributions



- Diffusion models → **Markov chain** for **denoising** intermediate states

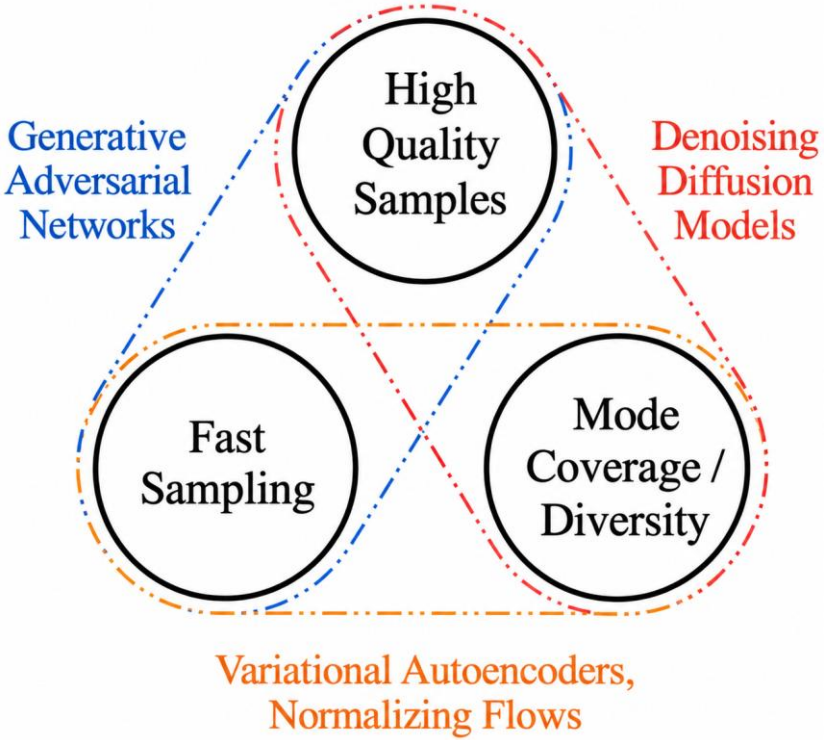
  Diffusion Video Slides → https://ducspe.github.io/DCPresentations/#diffusion



- **Transformers → we will focus our attention on them soon**

Coming Later

The Generative Learning Trilemma



Model Trade-offs:

Generative Adversarial Networks (GANs):

- **Strengths:** High-quality samples and fast generation.
- **Trade-off:** Poor mode coverage/diversity (prone to **mode collapse**).

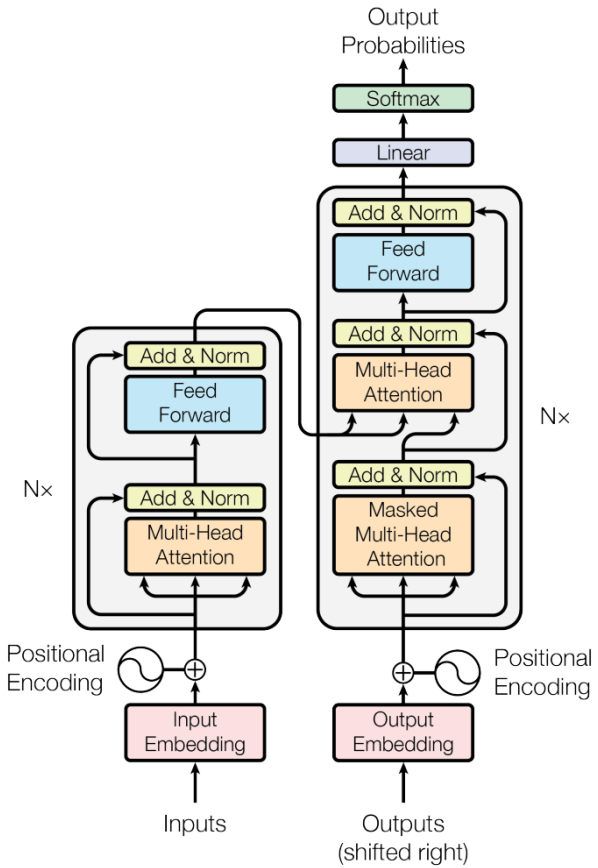
Denoising Diffusion Models:

- **Strengths:** High-quality samples and excellent diversity.
- **Trade-off:** Slow sampling speed.

Variational Autoencoders (VAEs) & Normalizing Flows:

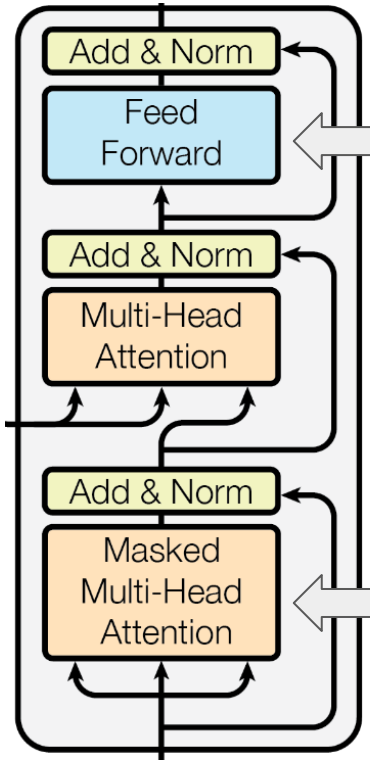
- **Strengths:** Fast sampling and good mode coverage/diversity.
- **Trade-off:** Lower overall sample quality (often blurry).

Transformers



“Attention is all you need” paper from 2017 by Vaswani et al.

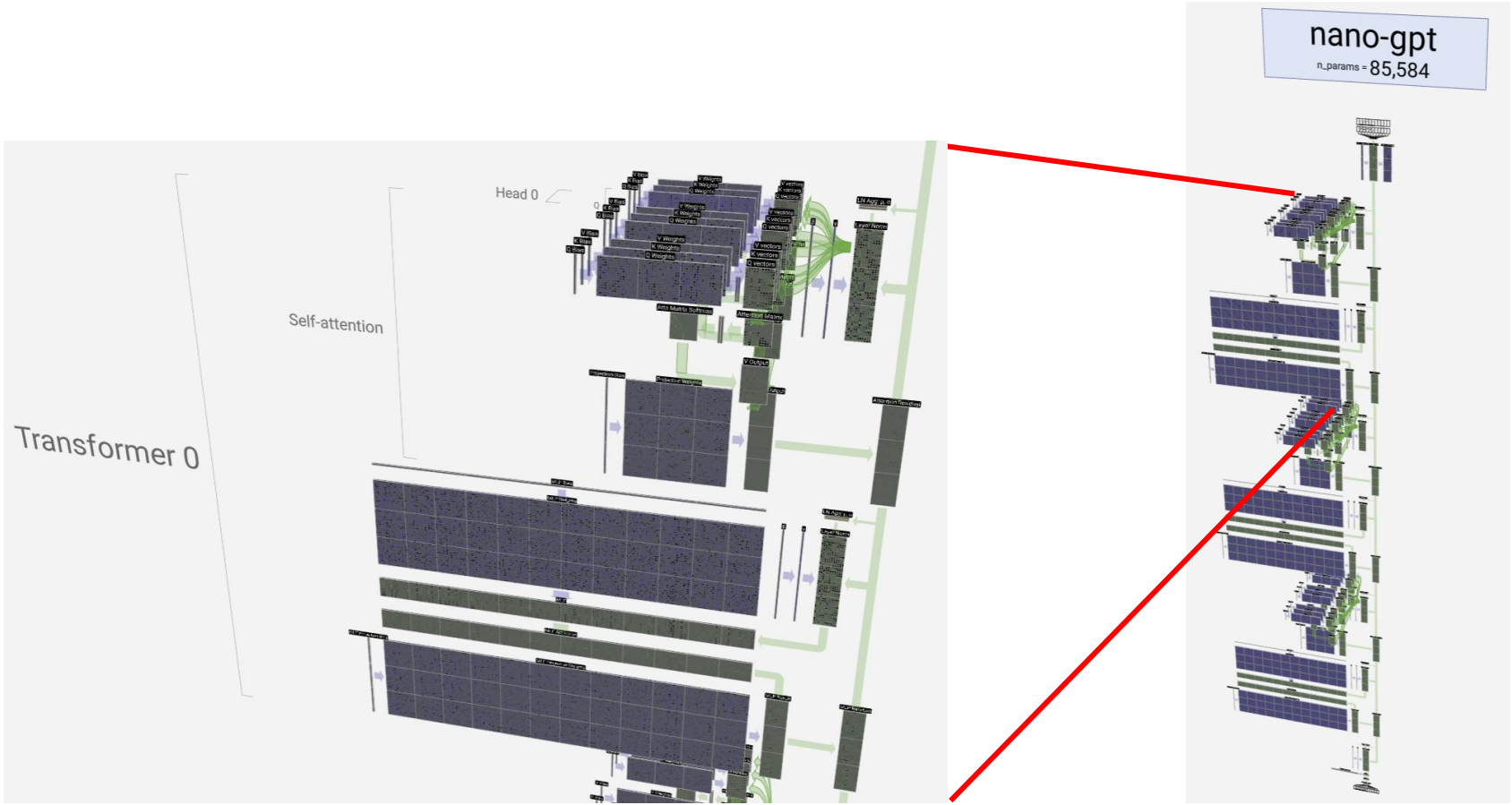
Conceptual Transformer Block



Secondly, a **Multilayer Perceptron (MLP)** crunches the data that was communicated among tokens in the previous self-attention phase.

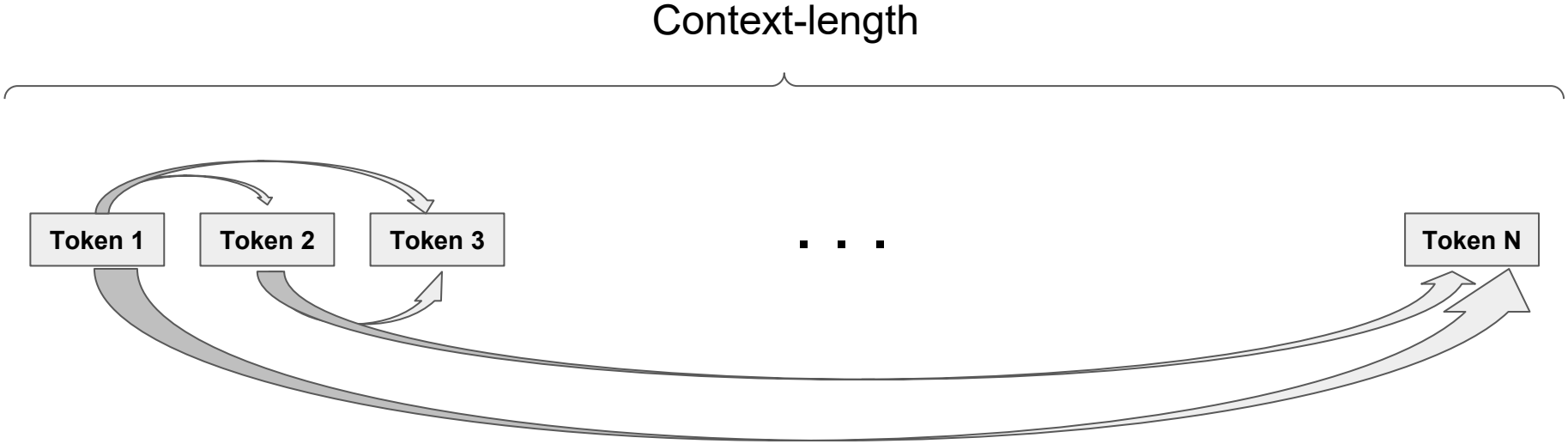
First, the tokens **communicate** between themselves / they attend to each other (**self-attention**).

Inside a GPT Block



Visualize the physical structure of an LLM interactively at → https://bbycroft.net/llm

Self-Attention



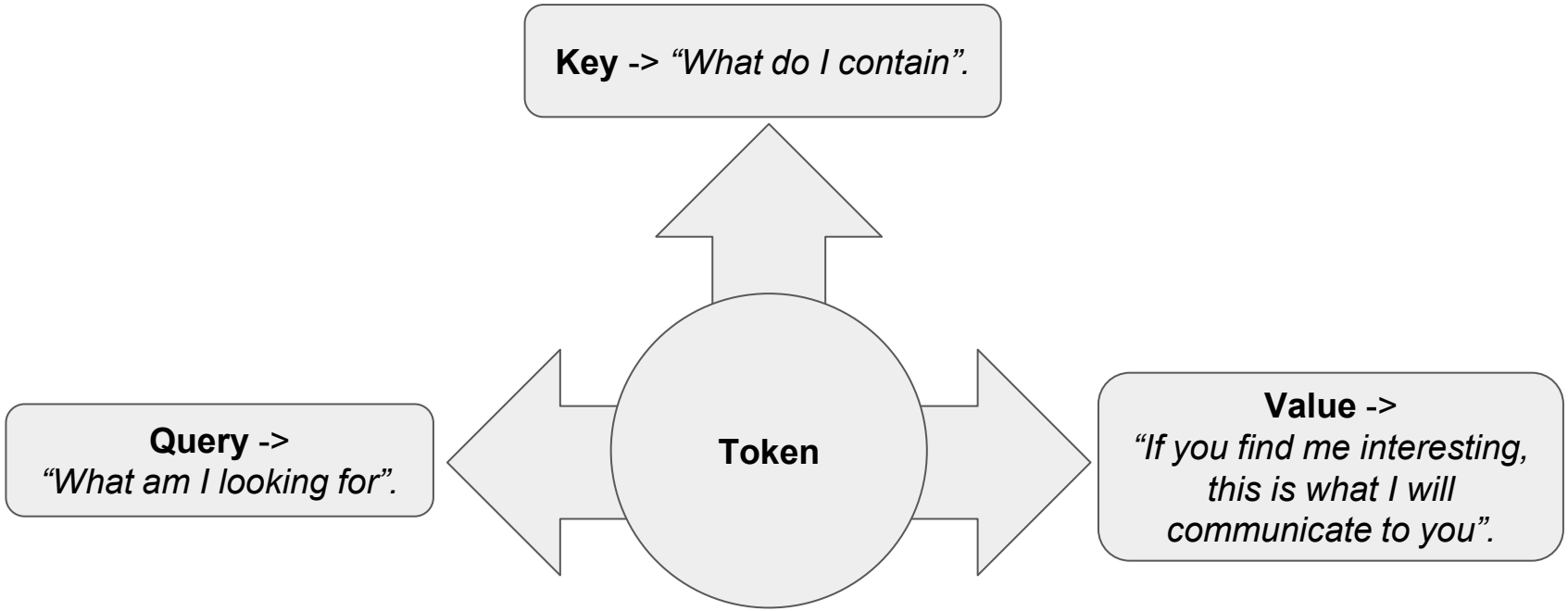
- **All tokens communicate with one another**.
- This is computationally expensive *[i.e. $O(N^2)$ complexity]* because **each token has to look at every other token** to compute an **attention score / attention weight**.

Increasing the Context Length

- **Sparse Attention** Mechanisms
- Linformer
- Reformer
- Longformer
- **Blockwise** Attention
- Ring Attention

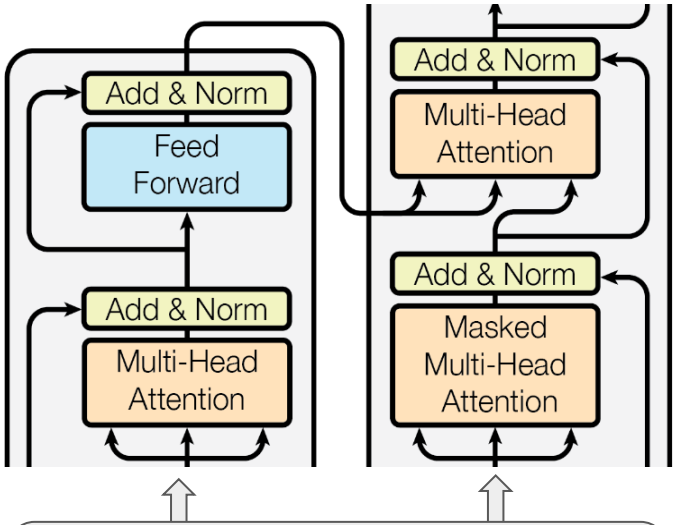
Reducing attention complexity is a very hot topic today 🔥 . This would allow for **more context memory** to be used when tackling problems. More memory means **longer chains of thinking** => **thinking is computationally distributed over more tokens**, which generally helps produce more accurate answers and can **reduce hallucinations**.

Key, Query and Value Embeddings



Self-Attention vs Cross-Attention

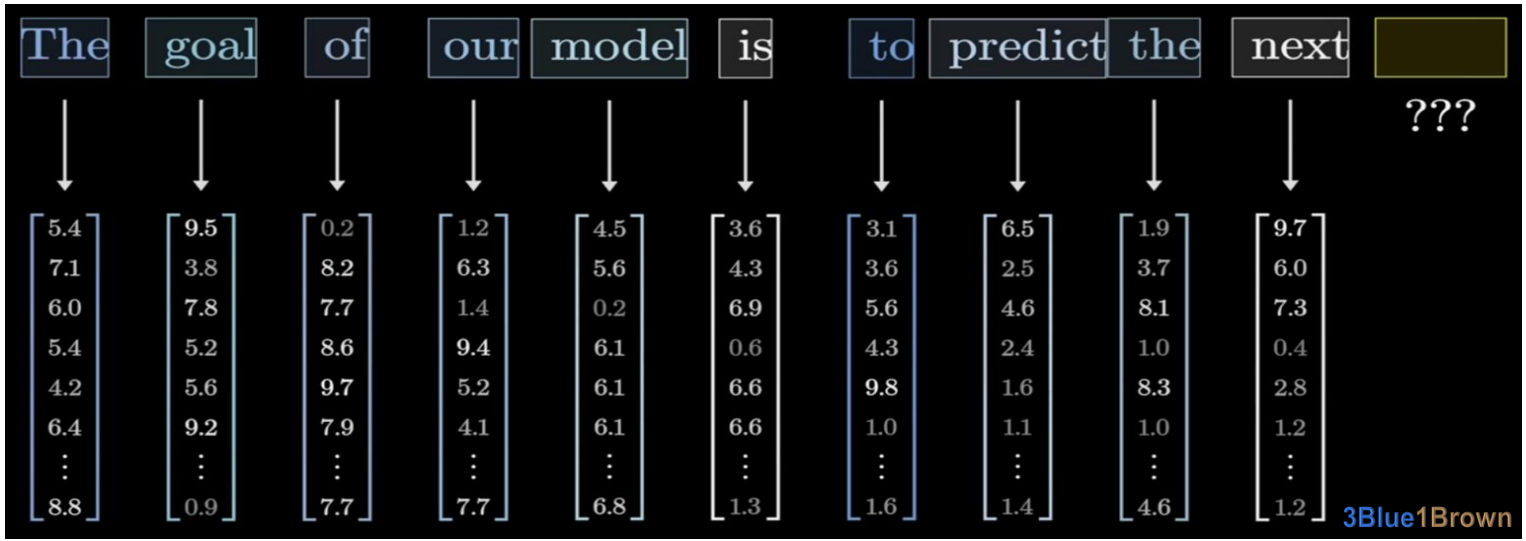
Cross-Attention -> the *queries* come from the **decoder**, whereas the *keys* and *values* are from the **encoder** side.



Self-attention -> the *key*, *query* and *value* vectors are related to the **same entity**, either the encoder, or the decoder.

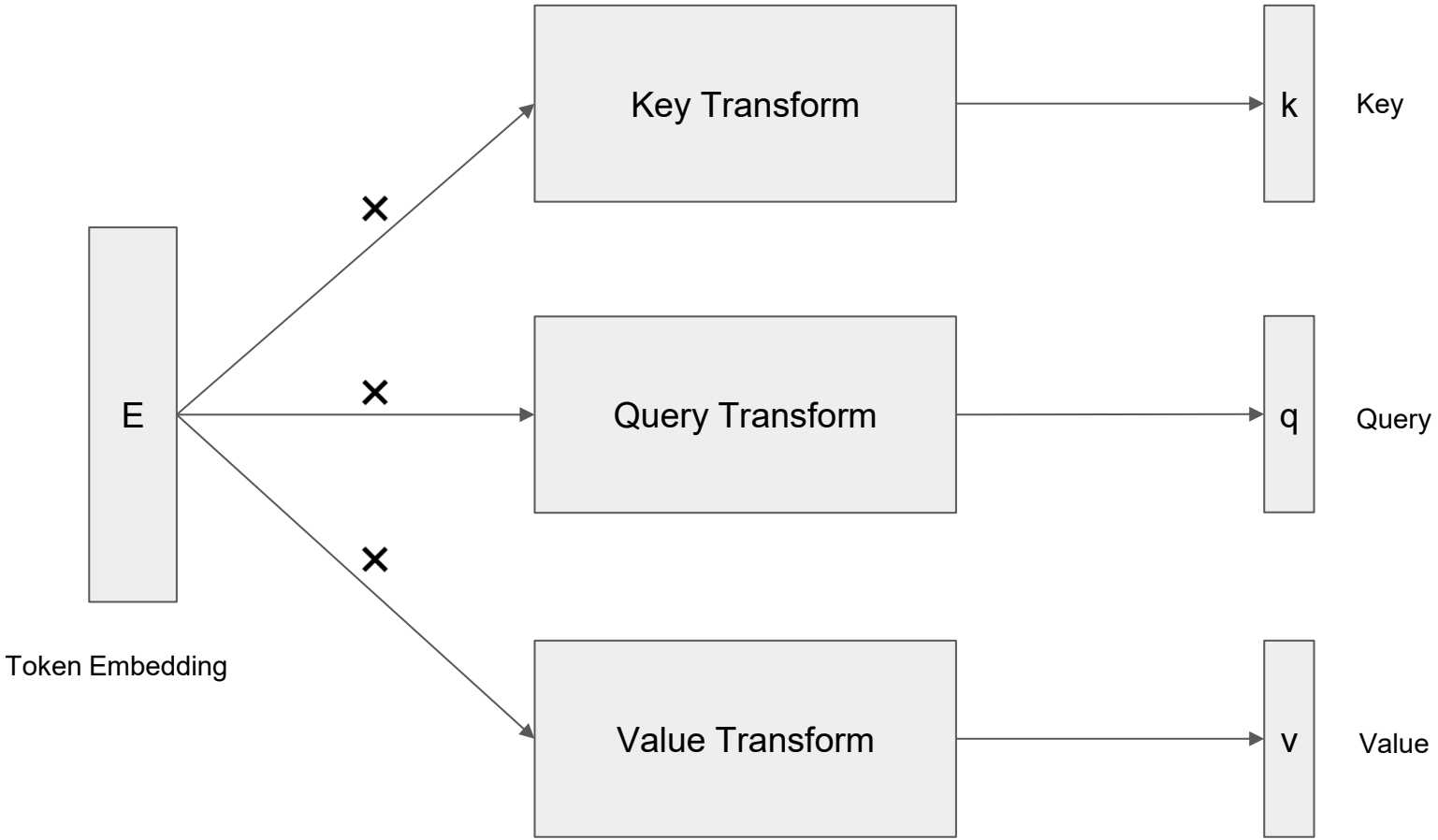
Tokens and their Embeddings

Tokens are mapped to embedding vectors in a semantic space using an embedding matrix. Our task is to predict the next token



In the practical session we will use **single-character tokens**. Watch these animated video slides about **sub-word** level **tokens** → https://ducspe.github.io/DCPresentations/#tokenization

Creating Keys, Queries, and Values



Transformation Matrices

$$\mathbf{E} = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_{\text{emb_dim}} \end{bmatrix}$$

$$\text{Key Matrix } \mathbf{W}_K = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1\,d_k} \\ w_{21} & w_{22} & \cdots & w_{2\,d_k} \\ \vdots & \vdots & \ddots & \vdots \\ w_{\text{emb_dim},1} & w_{\text{emb_dim},2} & \cdots & w_{\text{emb_dim},d_k} \end{bmatrix} \quad \mathbf{W}_K \in R^{\text{emb_dim} \times d_k} \qquad \mathbf{k} = \mathbf{E}\,\mathbf{W}_K$$

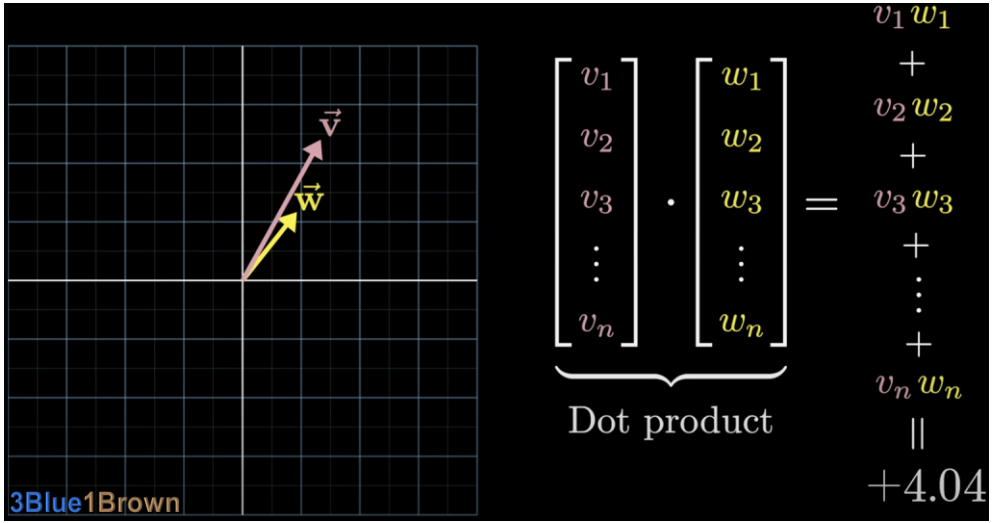
$$\text{Query Matrix } \mathbf{W}_Q = \begin{bmatrix} w'_{11} & w'_{12} & \cdots & w'_{1\,d_q} \\ w'_{21} & w'_{22} & \cdots & w'_{2\,d_q} \\ \vdots & \vdots & \ddots & \vdots \\ w'_{\text{emb_dim},1} & w'_{\text{emb_dim},2} & \cdots & w'_{\text{emb_dim},d_q} \end{bmatrix} \quad \mathbf{W}_Q \in R^{\text{emb_dim} \times d_q} \qquad \mathbf{q} = \mathbf{E}\,\mathbf{W}_Q$$

$$\text{Value Matrix } \mathbf{W}_V = \begin{bmatrix} \tilde{w}_{11} & \tilde{w}_{12} & \cdots & \tilde{w}_{1\,d_v} \\ \tilde{w}_{21} & \tilde{w}_{22} & \cdots & \tilde{w}_{2\,d_v} \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{w}_{\text{emb_dim},1} & \tilde{w}_{\text{emb_dim},2} & \cdots & \tilde{w}_{\text{emb_dim},d_v} \end{bmatrix} \quad \mathbf{W}_V \in R^{\text{emb_dim} \times d_v} \qquad \mathbf{v} = \mathbf{E}\,\mathbf{W}_V$$

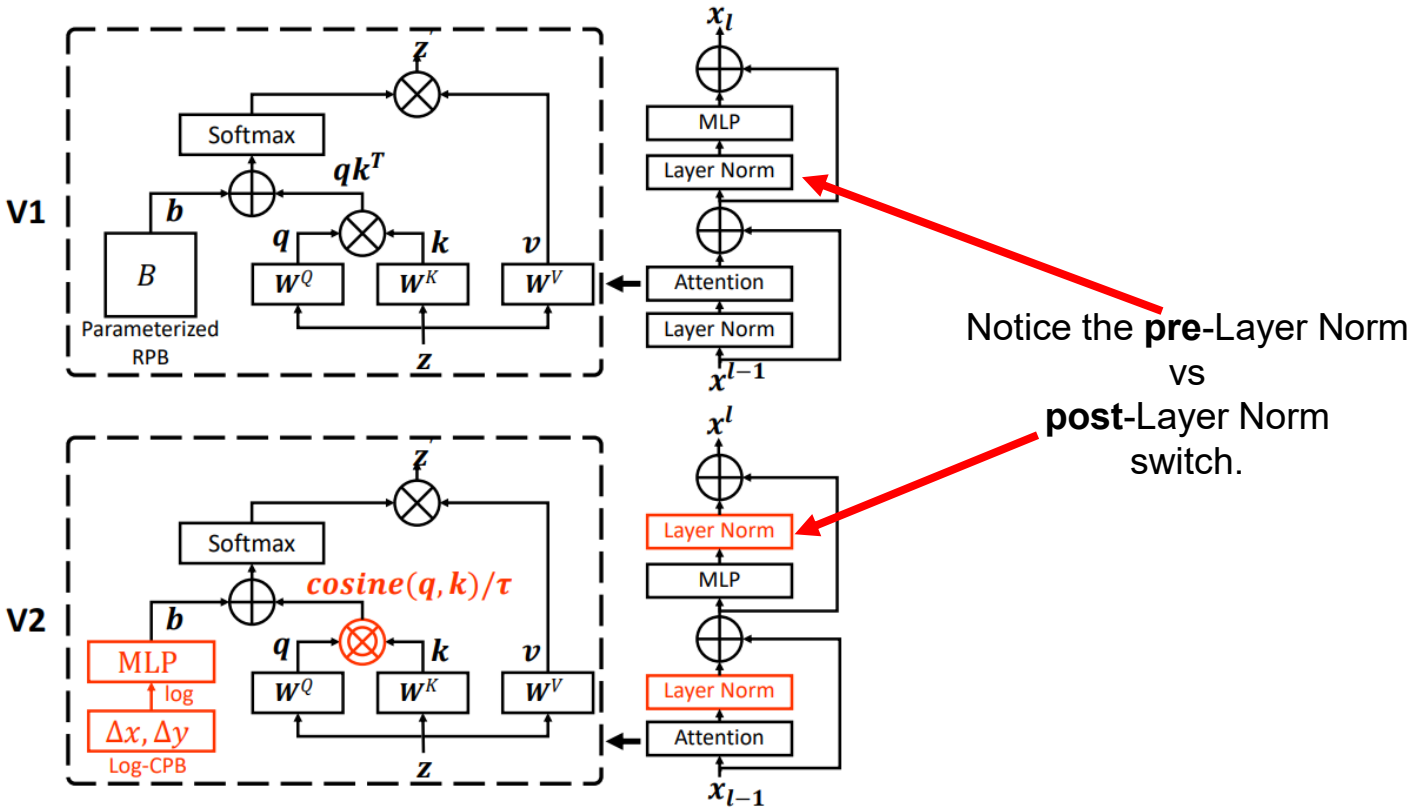
An embedding **transform** is a **multiplication** with a specialized **matrix**, the parameters of which the transformer needs to **learn**.

Measuring Similarity in Self-Attention

- The dot product **Query** · **Key** is the attention score, where **Query** and **Key** are **embedding vectors** derived from the token embedding via matrix transformations.
- Dot product measures **similarity between vectors** => **Attention** can be interpreted as the **alignment** between the **Key** and the **Query** vectors (i.e. two tokens find each other interesting).
- Instead of the dot product, other measures can be used, like the **cosine similarity** for example (see **Swin Transformer Version 2** paper).

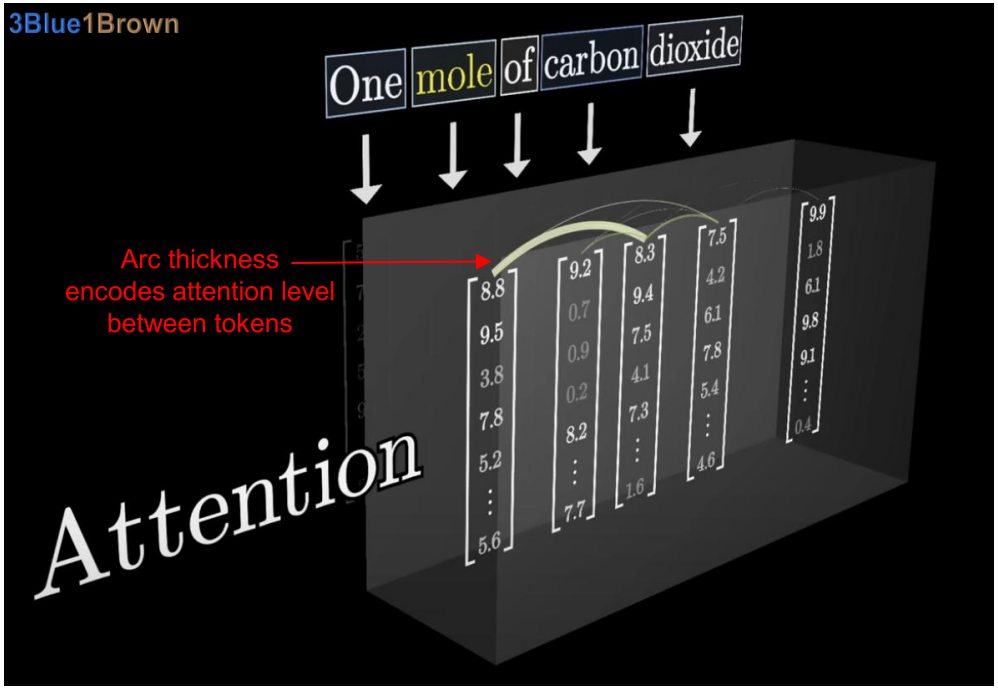


Cosine similarity



"Swin Transformer V2: Scaling Up Capacity and Resolution" by Liu et al.

How Context Changes Meaning

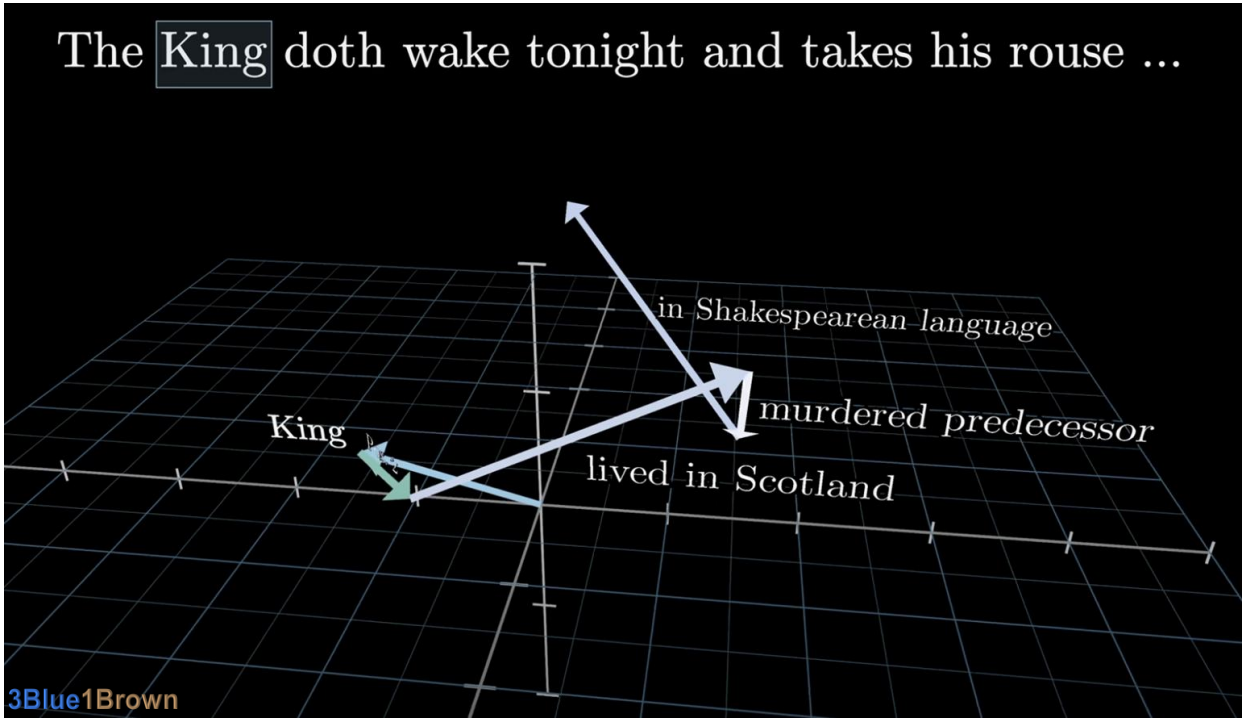


The **mole** on his face was benign.

The **mole** lives underground.

Attention will capture the meaning of a word **from the surrounding context**, using **weighted averages of neighboring embeddings**.

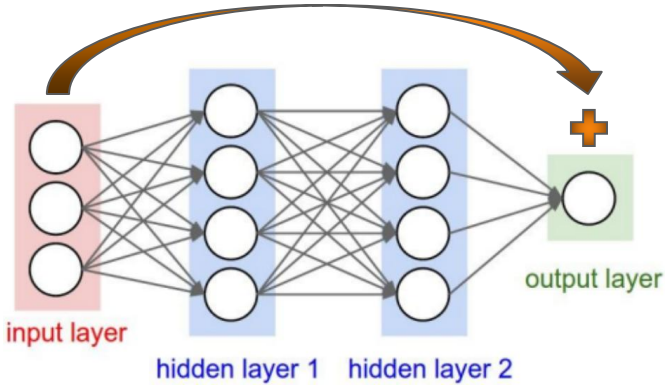
Navigating the Semantic Space via Attention



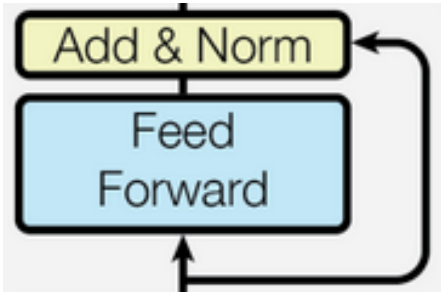
The embedding of the word “King” **absorbs via attention** the **embeddings from the other words** in Shakespeare's work, **as well as of the questions and statements that the users make** in their conversation with the Transformer model.

Computation Phase / Feed-Forward MLP

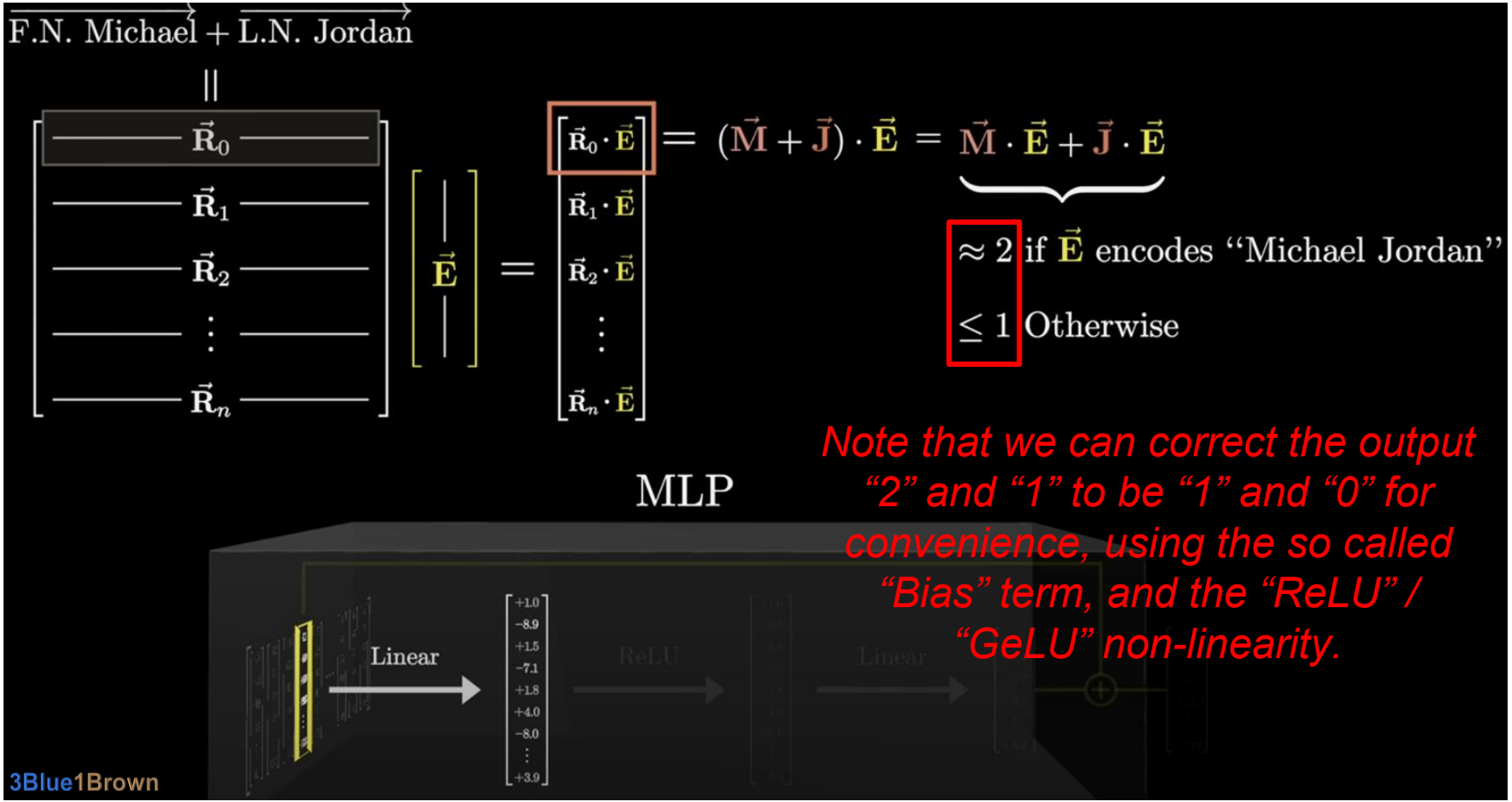
- After the communication between tokens is finished, an MLP has to **“think on what was said”** during the self-attention phase.
- This basically means that **new features are derived** after the communication phase **using a database of facts** encoded by the MLP.
- **Skip connections** can be viewed as **enriching the embedding**, but also helping the learning process from a technical perspective.
- **Normalization** is also very important for **stabilizing learning** and fostering convergence.



Let’s go back to what we have already seen and **notice the additional skip connection.**

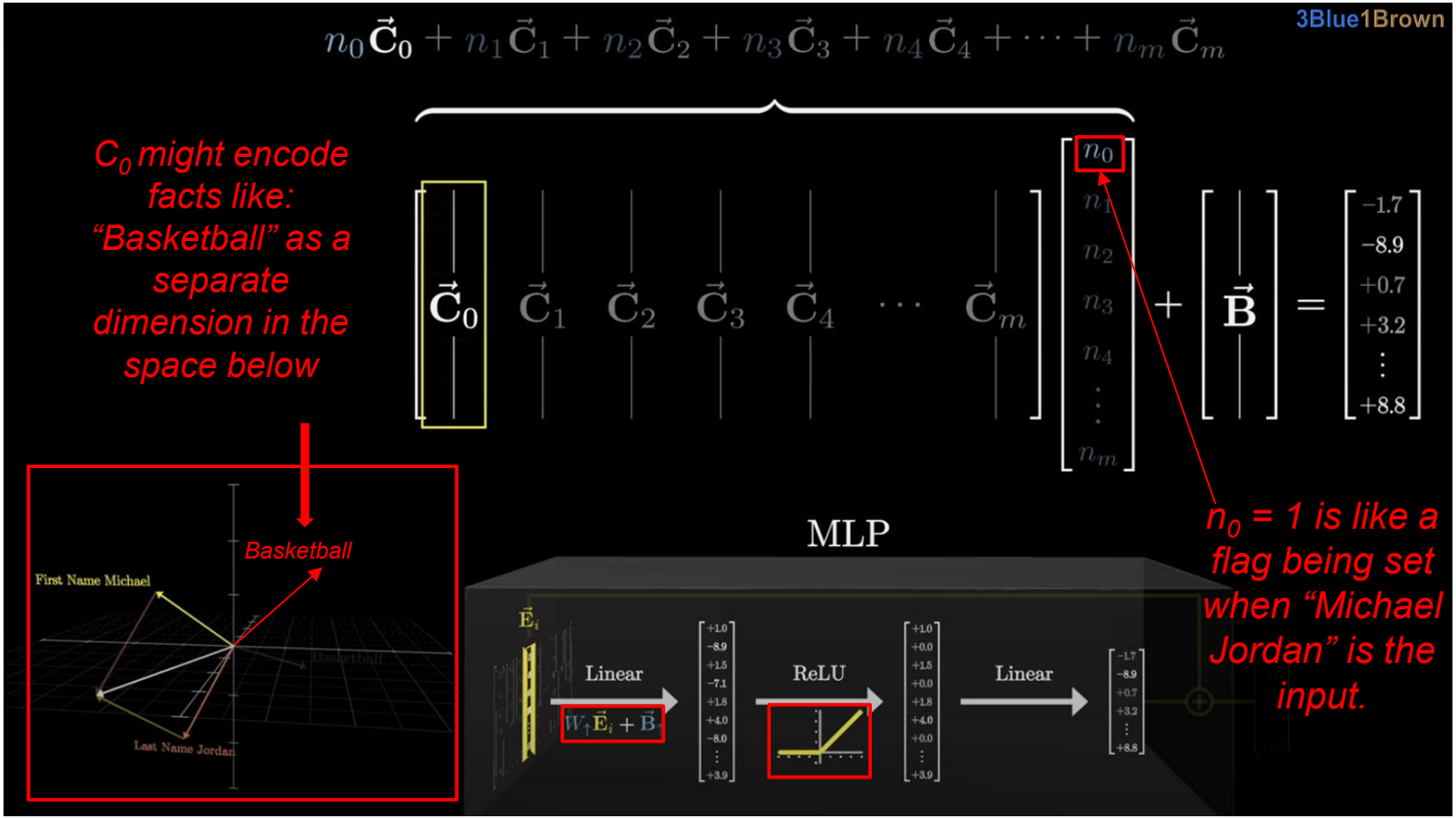


MLP Database of Facts



Note that we can correct the output "2" and "1" to be "1" and "0" for convenience, using the so called "Bias" term, and the "ReLU" / "GeLU" non-linearity.

MLP Database of Facts, Layer 2



“Grandmother cells” analogy in Neuroscience

Jennifer Aniston strikes a nerve

Single brain cells show selective response to specific celebrity photos.

Roxanne Khamsi

Is a single cell in your brain devoted to Jennifer Aniston or Bill Clinton? Maybe so, according to new research.

A recent experiment showed that single neurons in people's brains react to the faces of specific people. Researchers see the findings as evidence that our brains use fewer cells to decode a given image than previously thought.

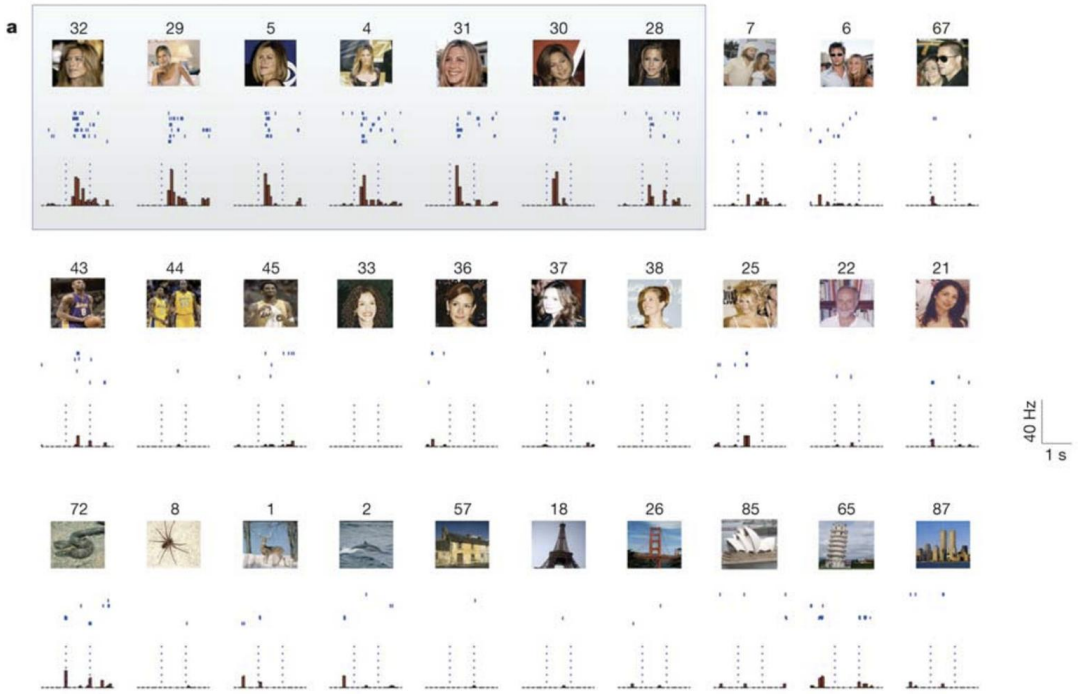
The subject of visual processing has sparked much scientific speculation in the past. Exactly how our brains extract meaning from an image remains unclear. At one end of the spectrum of possibilities, a network of cells would process various bits of information in a scene and piece it all together to form an understandable picture.



Snaps of Aniston made a single neuron in one subject’s brain light up

© *Photo/Lionel Cironneau*

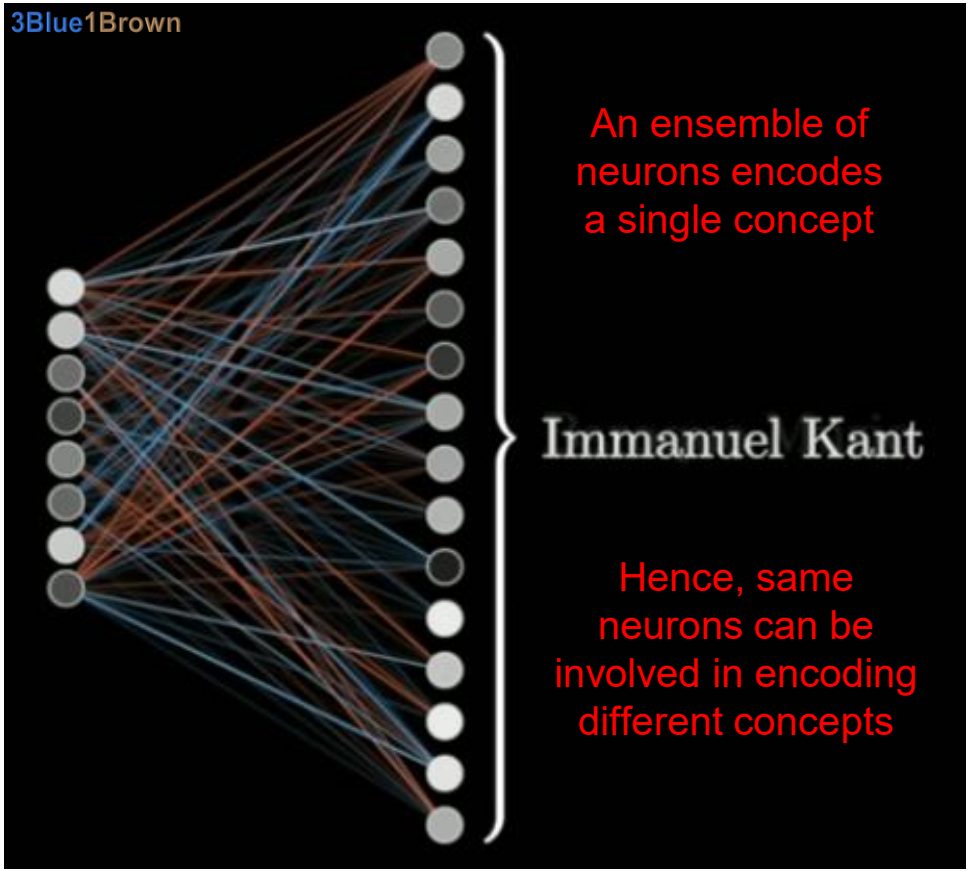
nature.com/news



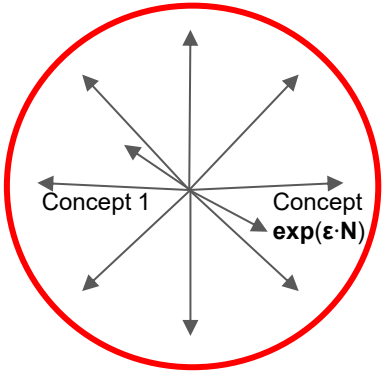
“**Invariant visual representation by single neurons in the human brain**”
R. Quian Quiroga et al., Nature 2005

“This unit was selectively activated by pictures of the actress Halle Berry as well as by a drawing of her”
“Notably, the unit was selectively activated by the letter string ‘Halle Berry’ ”

Superposition



A much more usual scenario is that an **ensemble of neurons encodes a concept**, rather than a single "grandmother cell". This leads to the principle of **superposition**, in which a lot of the **same neurons will be involved in encoding several concepts**. It's a **combinatorics game for semantics** that becomes **very expressive when we have very high dimensional spaces** like the 12,288 embedding size in the case of GPT-3 175B (Davinci) for example.



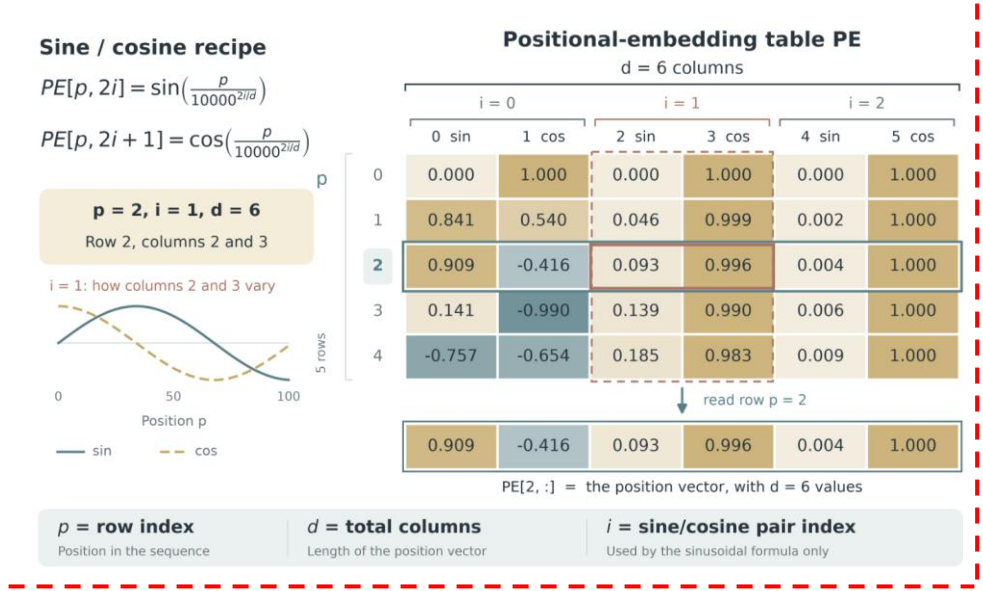
Positional Encoding

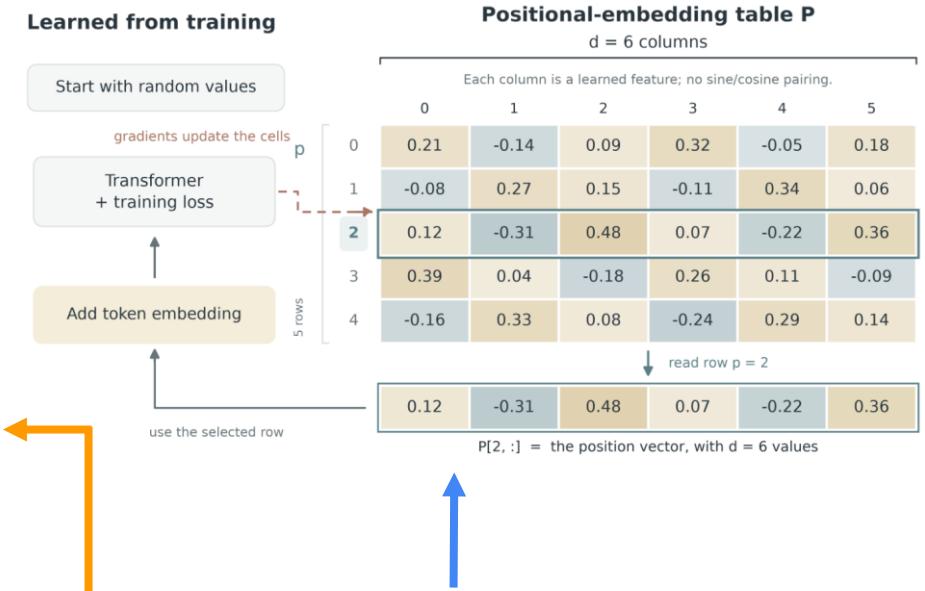


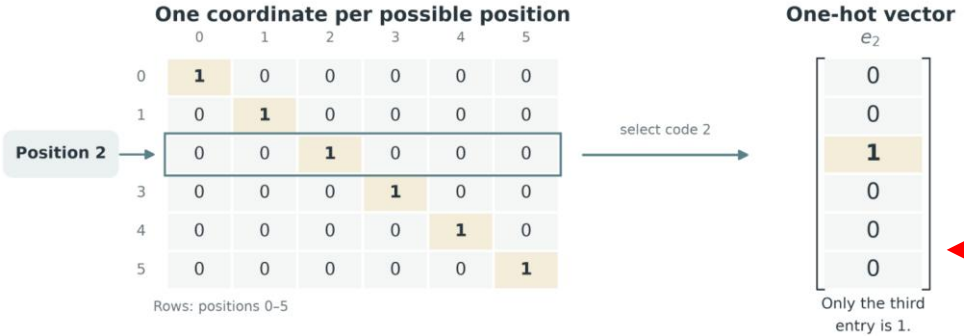
- The transformer treats the tokens as a **Bag of Words (BoW)**.
- We need to give each token a label that specifies its position in the form of a **counter ID**, for instance.
- It's noteworthy that the positional encoding information is **literally added** by a **"+"/ plus** operation.

There are various encoding schemes out there, such as **absolute encoding** or **relative encoding** for example. These seemingly small details can have a significant impact on how the transformer ends up performing. Check out the Swin Transformer paper for some empirical evidence.

Analytical vs Learned Positional Encoding





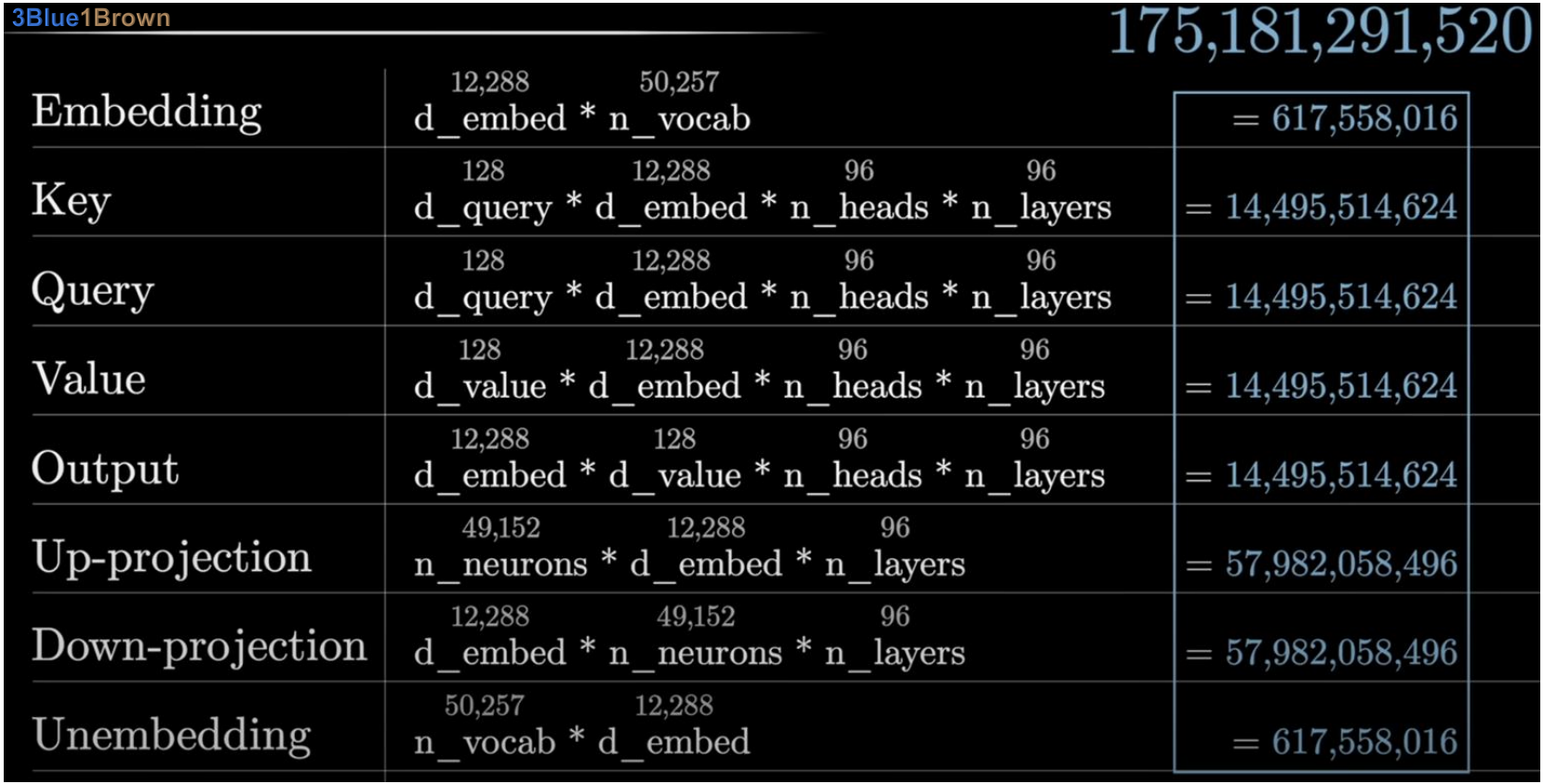


Positional encoding can be **learned during training**.

The original paper also used a fixed formula with **sines** and **cosines** to create positional embeddings.

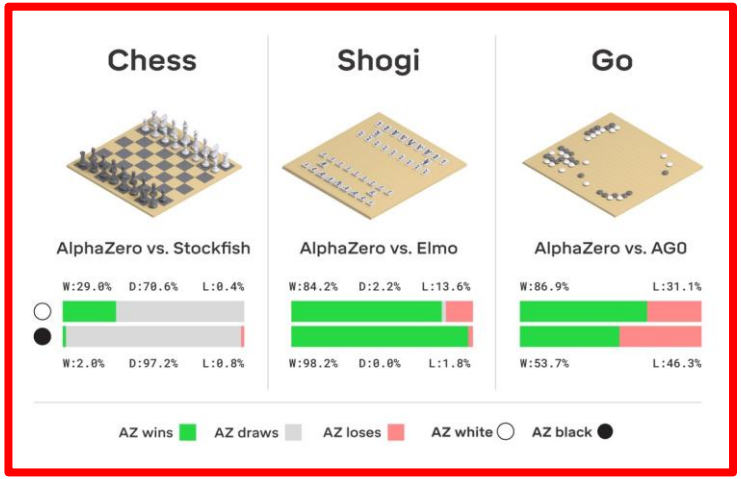
We **do not use the one-hot encoding**. This is a common misconception.

GPT-3 Tensors and Parameter Counts



Training an AI Assistant

1) Pretraining the **base model**.
2) Supervised Fine-Tuning (**SFT**).
3) Reward Modeling / **RM**.
4) Reinforcement Learning / **RL** (with human in the loop / human feedback).
5) Additional **Chain-of-Thought (CoT)** Reinforcement Learning.



DeepMind subsequently created **AlphaStar, AlphaFold, AlphaEvolve** using similar principles.

Conjecture: ChatGPT works so well, partly because it **borrowed** many **insights from** the **AlphaZero** games playing engine from back in 2017.

Words seem to be the new chess pieces that have to be smartly arranged.

https://deepmind.google/blog/alphazero-shedding-new-light-on-chess-shogi-and-go/

Pretraining the Foundational Model

- Use **raw data** to train a **Base Model**.
- The dataset is **huge** => potentially low quality in some places, but **very large quantity** (ex: from roughly 1 to 45 TB / Terabytes of text for the 2023 era).
- We obtain a **document completer** in the end, i.e. a very well informed and trained "auto-complete".
- **Thousands of GPUs** work in parallel (ex: ~2000 A100 GPUs in the 2023 era).

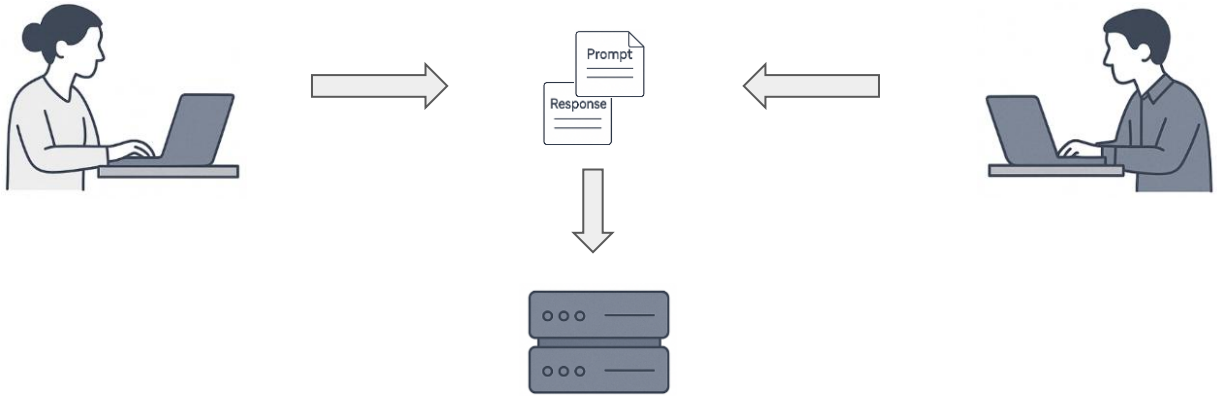


- Download LM Studio https://lmstudio.ai/
- Download models: **"Llama-3.2-1B"** and **"Llama-3.2-1B-Instruct"**.
- Load the **"Llama-3.2-1B"** base model.
- Eject the previous model and load the **"Llama-3.2-1B-Instruct"** model in a new chat session.
- What's the difference?

https://www.fz-juelich.de/en/ias/jsc/systems/supercomputers/**apply-for-computing-time**

Supervised Fine-Tuning (SFT)

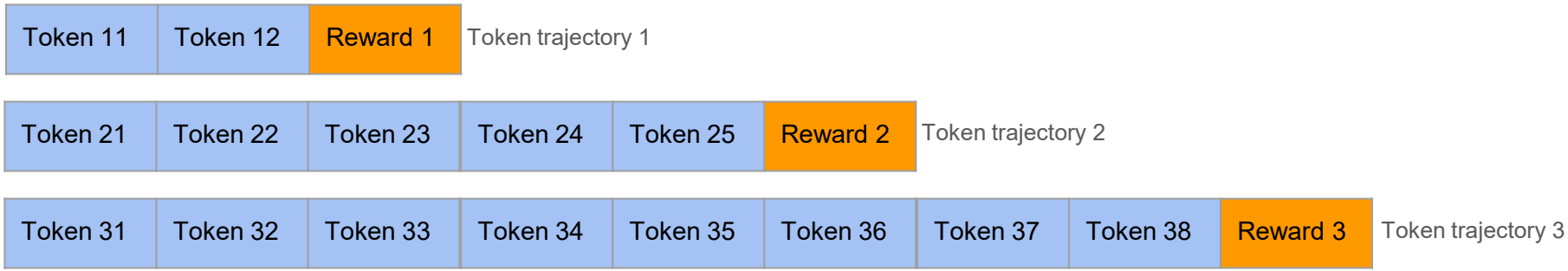
- **Low quantity, but very high quality data**: ~ 100K (prompt, response) tuples in the 2023 era.
- Domain Specialists / Contractors have to scrutinize the dataset so that close to ideal **(prompt, response) tuples** are assembled.
- Fewer GPUs are required than in step 1 (ex: 1-100 in the 2023 era).
- The outcome is the so-called **SFT model**.



[The figure sub-components were generated using the OpenAI diffusion model]

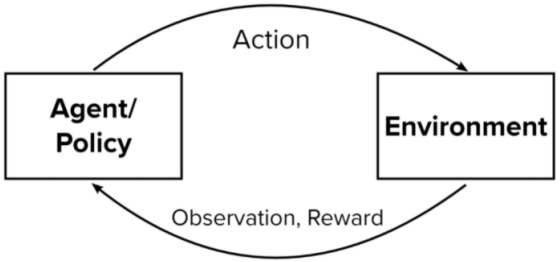
Reward Modeling

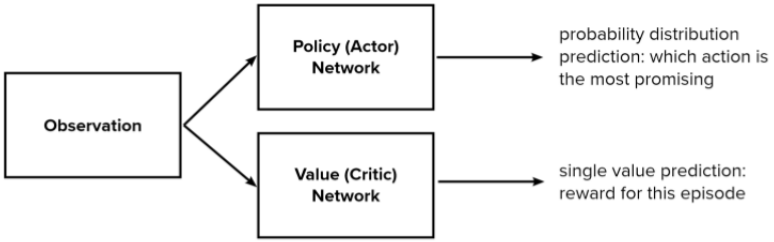
- Ask the SFT model to produce **multiple answers** per prompt.
- Ask human contractors to carefully **rank these answers**.
- Train a reward model on these rankings.
- Requires on the order of 1 to 100 GPUs for training (2023 era figures).
- The outcome is the so-called **Reward Model / RM** => **Evaluates token trajectories.**



Reinforcement Learning from Human Feedback (RLHF)

- Train the model using **Proximal Policy Optimization (PPO)**.
- Use the previously trained reward model to **evaluate the reward**.
- PPO will have the task of generating **token "trajectories"** that will have a **very good overall score**.
- Requires on the order of 1 to 100 GPUs for training (2023 era figures).
- The outcome is the so called **RLHF model** (reinforcement learning with **human feedback**).

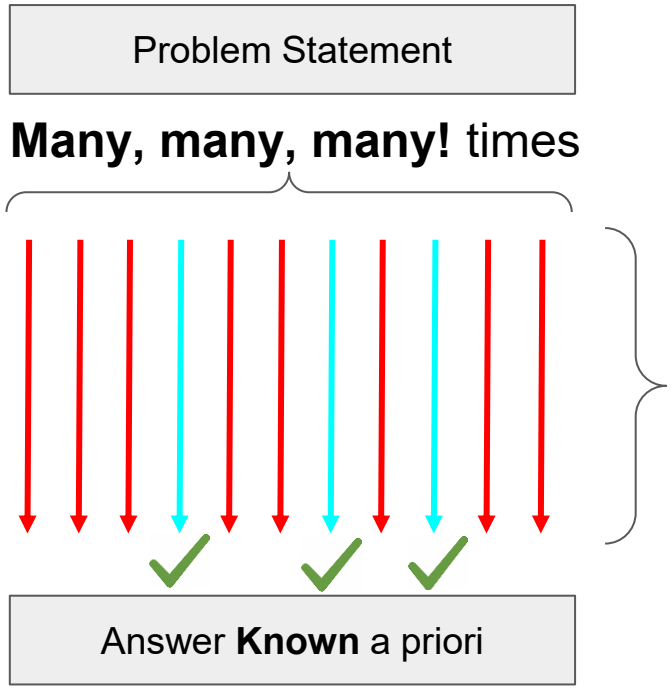




Note that the RL aspect has grown significantly since the earlier stages of RLHF in 2023.

Chain-Of-Thought Reasoning

To generate the recent **chain-of-thought** reasoning systems, we need **even more Reinforcement Learning**, i.e. something more and **different** than RLHF.



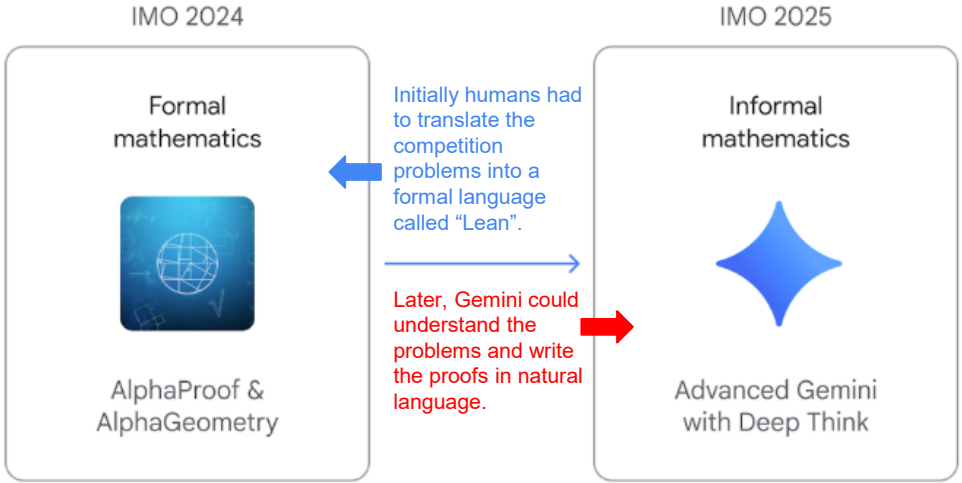
The model tries different token trajectories and **receives rewards** based on their correctness. Training updates its parameters to make **successful reasoning more likely on future problems.**

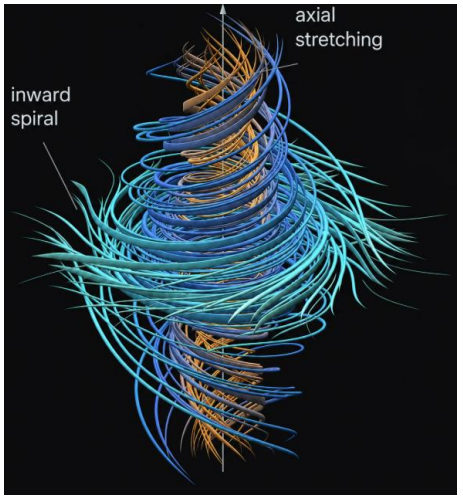
One interesting aspect is that training on coding tasks seems to generally improve logic ability across other areas such as math and sciences.

Reinforcement Learning with Verifiable Rewards (RLVR)

Some domains like math and coding are especially amenable to reinforcement learning because we can create a **clear and verifiable reward signal**.

This has contributed to significant advances in **agentic coding** and **mathematical reasoning**, with recent milestones including proofs of International Mathematical Olympiad problems and even a proposed solution to the **Navier–Stokes Millennium Prize Problem** in September 2026.

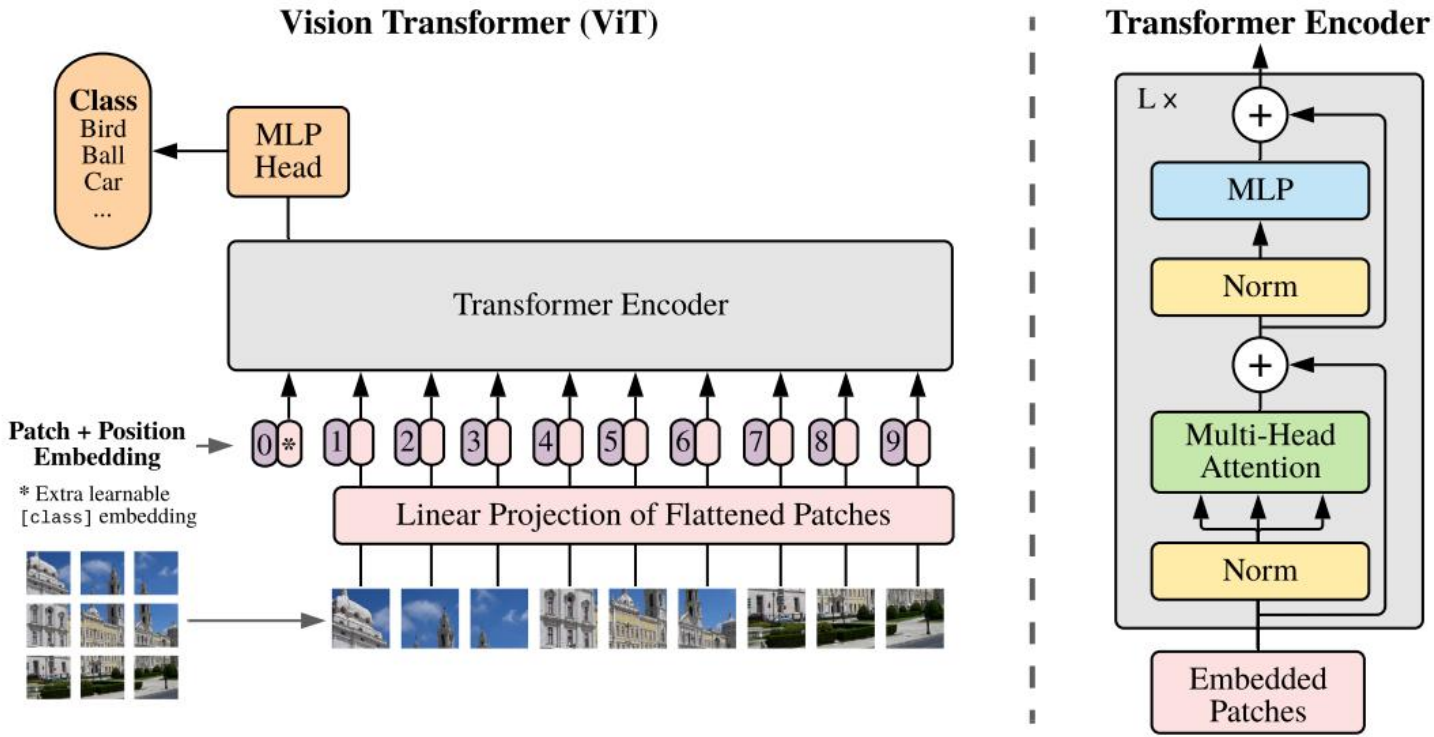




https://deepmind.google/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-mathematical-olympiad/

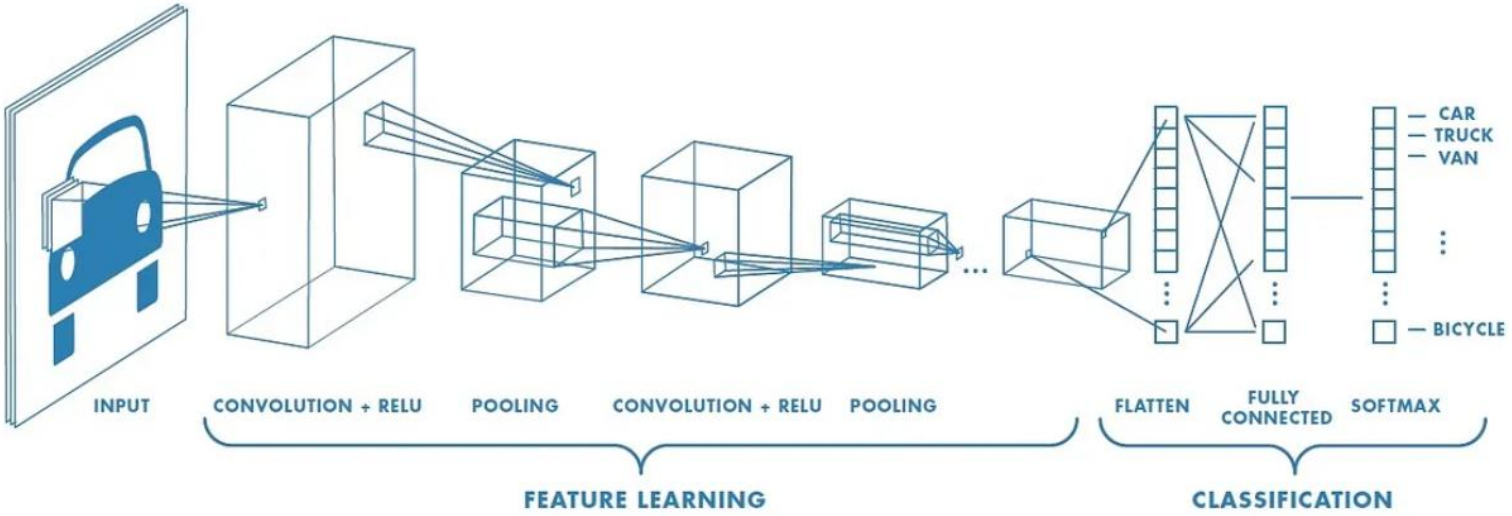
https://openai.com/index/navier-stokes-solution/

Vision Transformer / ViT



“An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale” by A. Dosovitskiy et al. (2021)

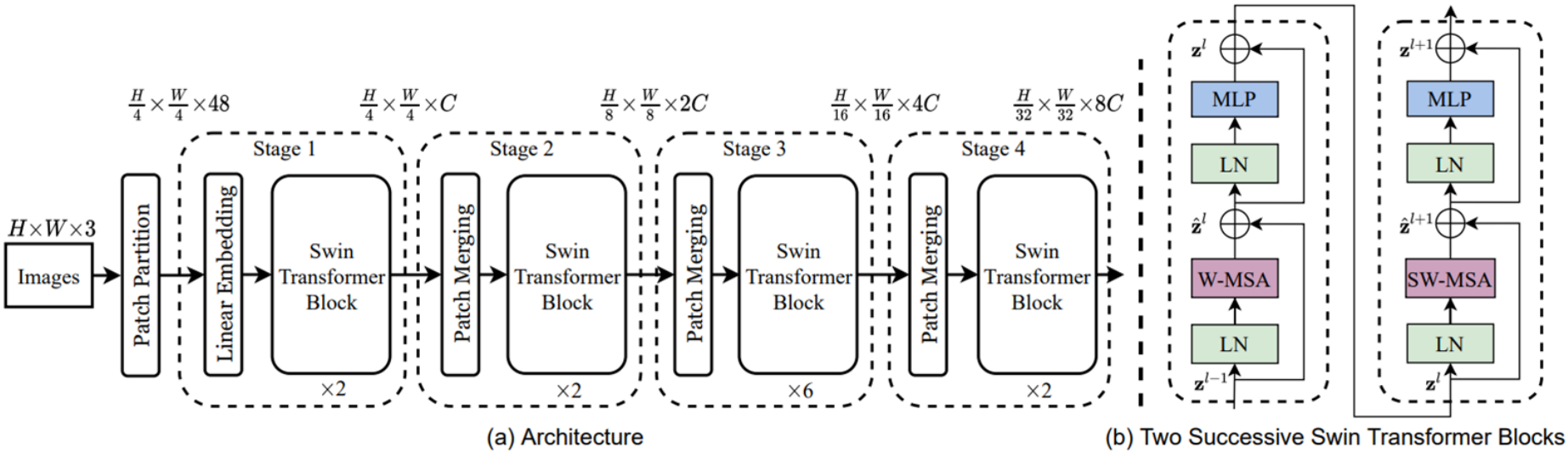
Convolutional Neural Networks (CNNs)



- Generally applied in **computer vision** tasks, to extract **visual features** such as horizontal **edges**, vertical edges, edges of different angles, as well as more **complex shapes** like squares, circles, etc.
- CNNs are well suited to **parallel computation** on GPUs, allowing them to be efficiently integrated into a Transformer pipeline.
- Convolutional layers can be used at the **input stage of a vision transformer** to extract visual features. These feature maps are then converted into a **sequence of token embeddings**, making the subsequent processing **similar to the natural language processing scenario**.

Visualize interactive demo at -> https://cs231n.github.io/convolutional-networks

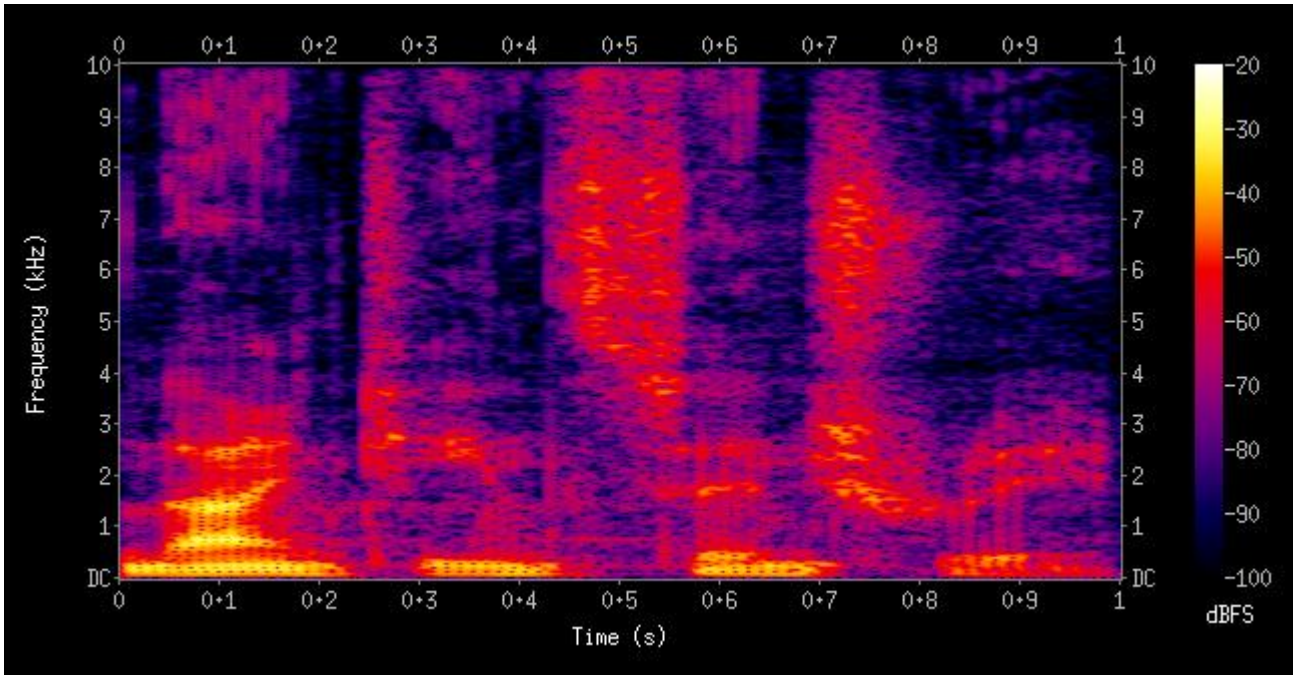
Swin Transformer



“Swin Transformer: Hierarchical Vision Transformer using Shifted Windows” by Liu et al. (2021)

Audio Transformer

Raw sound waves can be mapped to a different space using **STFT** (Short-time Fourier Transform).



Here, time series become images => **problem adapted** to be tackled by a ViT / Swin Transformer.

Time Series Transformer (TST)

Time-series values are converted into embeddings for processing by the Transformer; quantization into discrete tokens is optional.

Implementations available at -> ***https://huggingface.co/docs/transformers/model_doc/time_series_transformer***

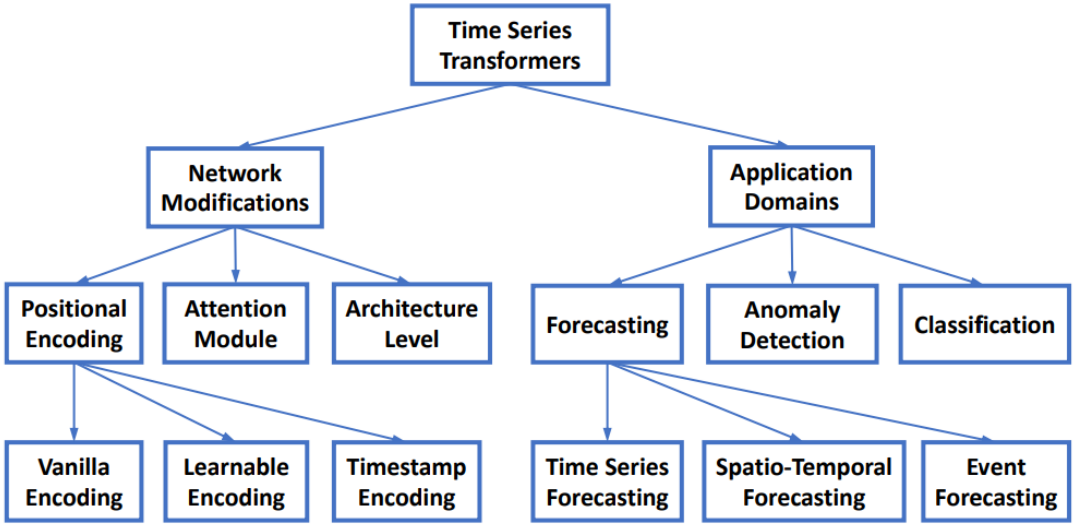
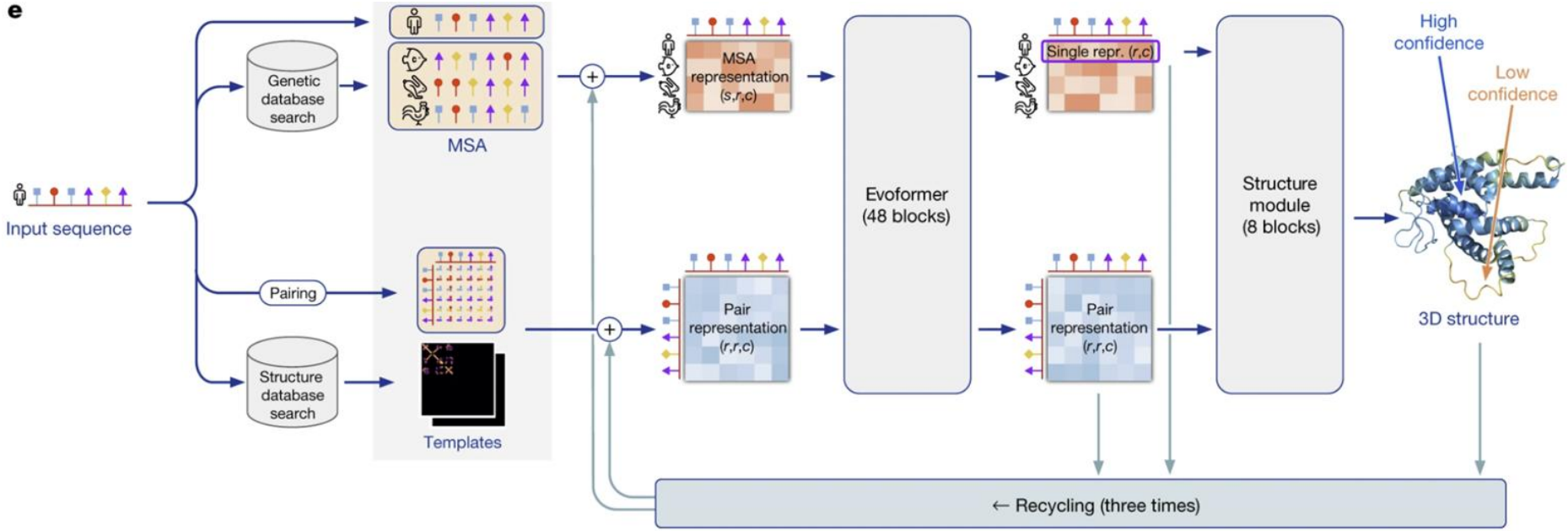


Figure 1: Taxonomy of Transformers for time series modeling from the perspectives of network modifications and application domains.

"Transformers in Time Series: A Survey" paper, by Wen et al. (2023)

AlphaFold2: Graph-Based Transformers and Geometric Attention



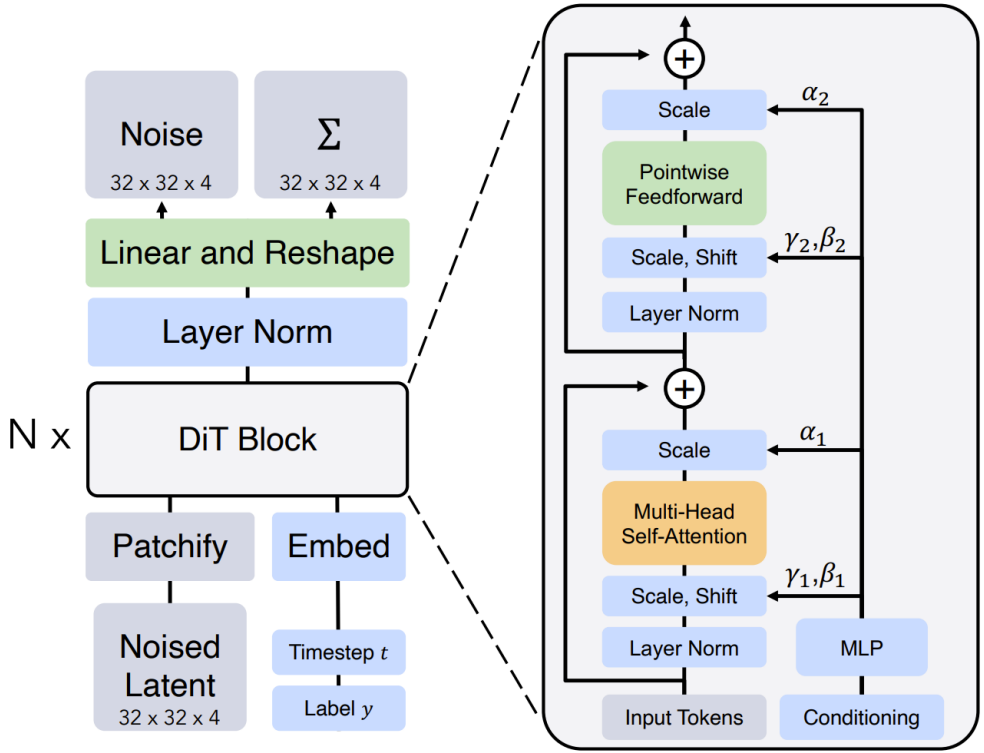
"Highly accurate protein structure prediction with AlphaFold" by John Jumper et al. (Nature, 2021)

- **AlphaFold2** offers many insights on using **Transformers in** relatively **data-scarce regimes**.
- It is also very insightful for cases where you need to **handle 1D, 2D and 3D data formats** within a single pipeline.
- It's not just a transformer, it's a hybrid system in conjunction with a **combination of many subtle training tricks** to make protein structure prediction very accurate in the end, acceptable enough for real-world laboratory experiments.

AlphaFold2 Video Slides → https://ducspe.github.io/DCPresentations/#alphafold2

https://nazimbouatta.scholars.harvard.edu/lectures

Latent Diffusion Transformer



Latent Diffusion Transformer

DiT Block with adaLN-Zero

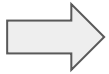
William Peebles and Saining Xie. Scalable Diffusion Models with Transformers. ICCV, 2023.

Diffusion Video Slides → https://ducspe.github.io/DCPresentations/#diffusion

Building Attention 1: Averaging with Loops

```
# Version 1:
# We want x[b,t] = mean_{i<=t} x[b,i]
xbow = torch.zeros((B,T,C))
for b in range(B):
    for t in range(T):
        xprev = x[b,:t+1] # (t,C)
        xbow[b,t] = torch.mean(xprev, 0)

print(x[0])
print(xbow[0])
```



```
torch.Size([4, 8, 2])

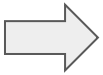
tensor([[ 1.9269,  1.4873],
        [ 0.9007, -2.1055],
        [ 0.6784, -1.2345],
        [-0.0431, -1.6047],
        [-0.7521,  1.6487],
        [-0.3925, -1.4036],
        [-0.7279, -0.5594],
        [-0.7688,  0.7624]])

tensor([[ 1.9269,  1.4873],
        [ 1.4138, -0.3091],
        [ 1.1687, -0.6176],
        [ 0.8657, -0.8644],
        [ 0.5422, -0.3617],
        [ 0.3864, -0.5354],
        [ 0.2272, -0.5388],
        [ 0.1027, -0.3762]])
```

Building Attention 2: Matrix Multiplication

```
# Version 1
# We want x[b,t] = mean_{i<=t} x[b,i]
xbow = torch.zeros((B,T,C))
for b in range(B):
   for t in range(T):
       xprev = x[b,:t+1] # (t,C)
       xbow[b,t] = torch.mean(xprev, 0)
print(x[0])
print(xbow[0])

# Version 2
wei = torch.tril(torch.ones(T, T))
wei = wei / wei.sum(1, keepdim=True)
xbow2 = wei @ x # (B, T, T) @ (B, T, C) ->(B, T, C)
print("Are xbow and xbow2 the same? -> ",
torch.allclose(xbow, xbow2))
```



```
torch.Size([4, 8, 2])

tensor([[ 1.9269,  1.4873],
        [ 0.9007, -2.1055],
        [ 0.6784, -1.2345],
        [-0.0431, -1.6047],
        [-0.7521,  1.6487],
        [-0.3925, -1.4036],
        [-0.7279, -0.5594],
        [-0.7688,  0.7624]])

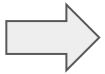
tensor([[ 1.9269,  1.4873],
        [ 1.4138, -0.3091],
        [ 1.1687, -0.6176],
        [ 0.8657, -0.8644],
        [ 0.5422, -0.3617],
        [ 0.3864, -0.5354],
        [ 0.2272, -0.5388],
        [ 0.1027, -0.3762]])

Are xbow and xbow2 the same? ->  True
```

Building Attention 3: Masking and Softmax

```
# Version 2
wei = torch.tril(torch.ones(T, T))
wei = wei / wei.sum(1, keepdim=True)
xbow2 = wei @ x # (B, T, T) @ (B, T, C) ->(B, T, C)
print("Are xbow and xbow2 the same? -> ",
torch.allclose(xbow, xbow2))

# Version 3: using Softmax
tril = torch.tril(torch.ones(T,T))
wei = torch.zeros((T,T))
wei = wei.masked_fill(tril == 0, float('-inf'))
wei = F.softmax(wei, dim=-1)
xbow3 = wei @ x
print("Are xbow/xbow2 equal to xbow3? -> ",
torch.allclose(xbow, xbow3))
```



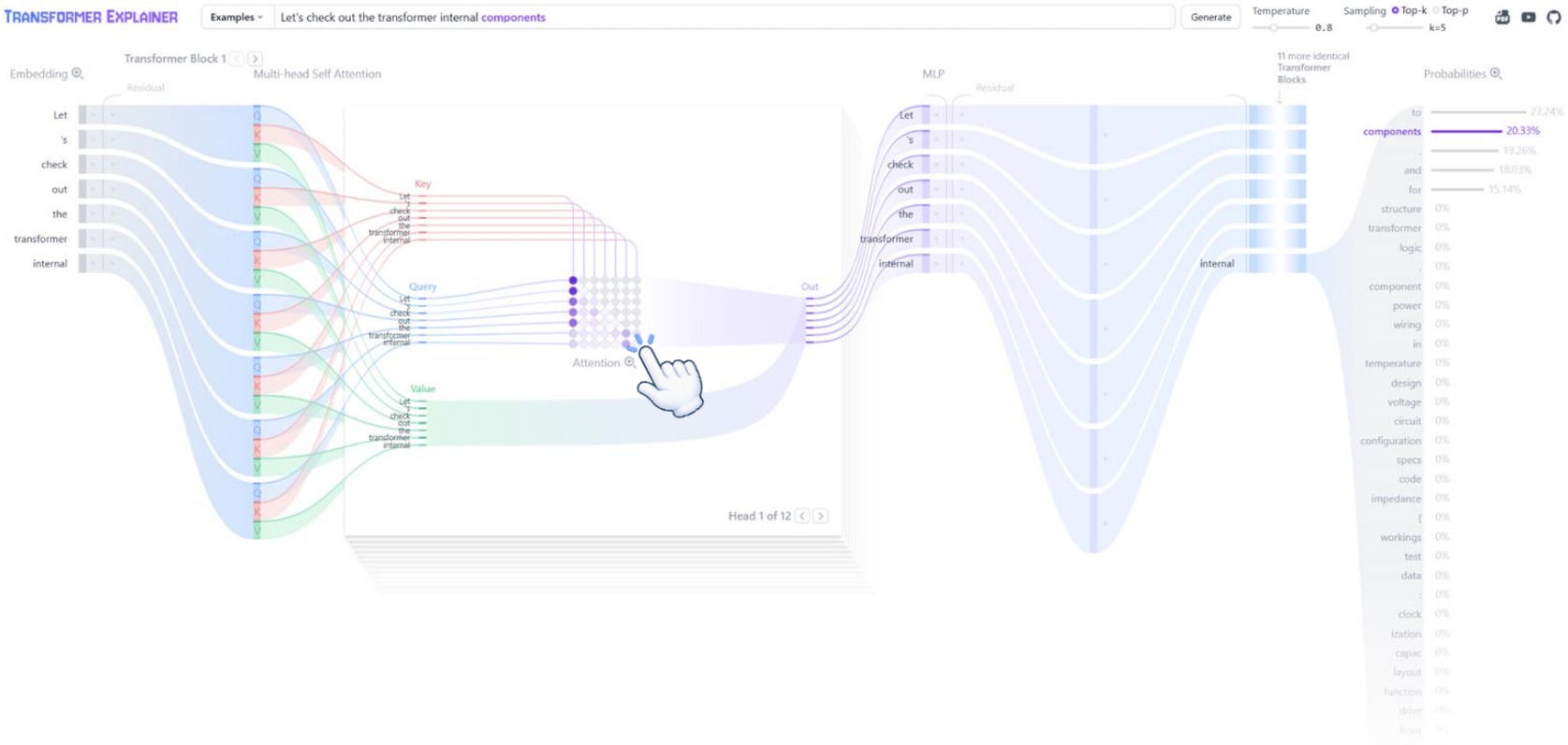
```
torch.Size([4, 8, 2])

tensor([[ 1.9269,  1.4873],
        [ 0.9007, -2.1055],
        [ 0.6784, -1.2345],
        [-0.0431, -1.6047],
        [-0.7521,  1.6487],
        [-0.3925, -1.4036],
        [-0.7279, -0.5594],
        [-0.7688,  0.7624]])

tensor([[ 1.9269,  1.4873],
        [ 1.4138, -0.3091],
        [ 1.1687, -0.6176],
        [ 0.8657, -0.8644],
        [ 0.5422, -0.3617],
        [ 0.3864, -0.5354],
        [ 0.2272, -0.5388],
        [ 0.1027, -0.3762]])

Are xbow/xbow2 equal to xbow3? ->  True
```

Let’s make the theory more tangible



Try clicking on things via this interactive tool and relate to what we discussed → ***https://poloclub.github.io/transformer-explainer/***

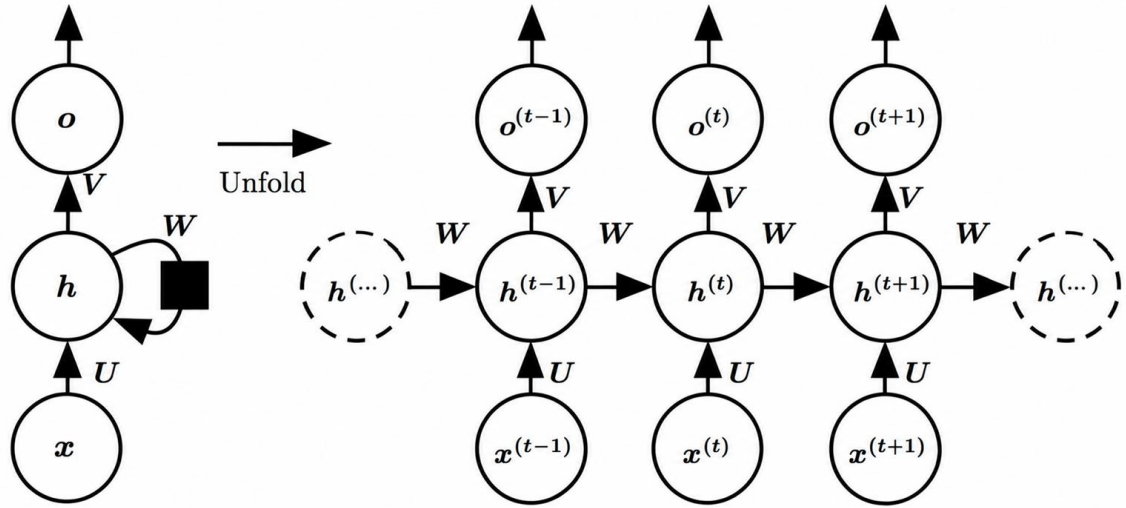
Note that if you wish to type your own sentences in the top field, you’ll have to wait a while for the browser to download GPT-2 in the background.

Takeaways

- Generative AI is the **art of encoding complex real world distributions**, such that we can **generate creative results** later **via sampling** from the learned distribution.
- Transformers are powerful neural networks that **borrow the best ideas** from prior models in the AI ecosystem and **combine them together for a synergistic effect**.
- **Self-attention** and **Feed-Forward MLP** are the major conceptual components of a **Transformer block**.
- **Self-attention** is essentially a **communication graph** where tokens exchange **information stored in channels** among themselves.
- The **Feed-Forward MLP** is the memory database of the transformer, used to enrich the input embeddings with new facts.
- **Residual additive connections** and **pre- / post-normalization** are important details / attributes, that contribute towards a more successful training and faster convergence.

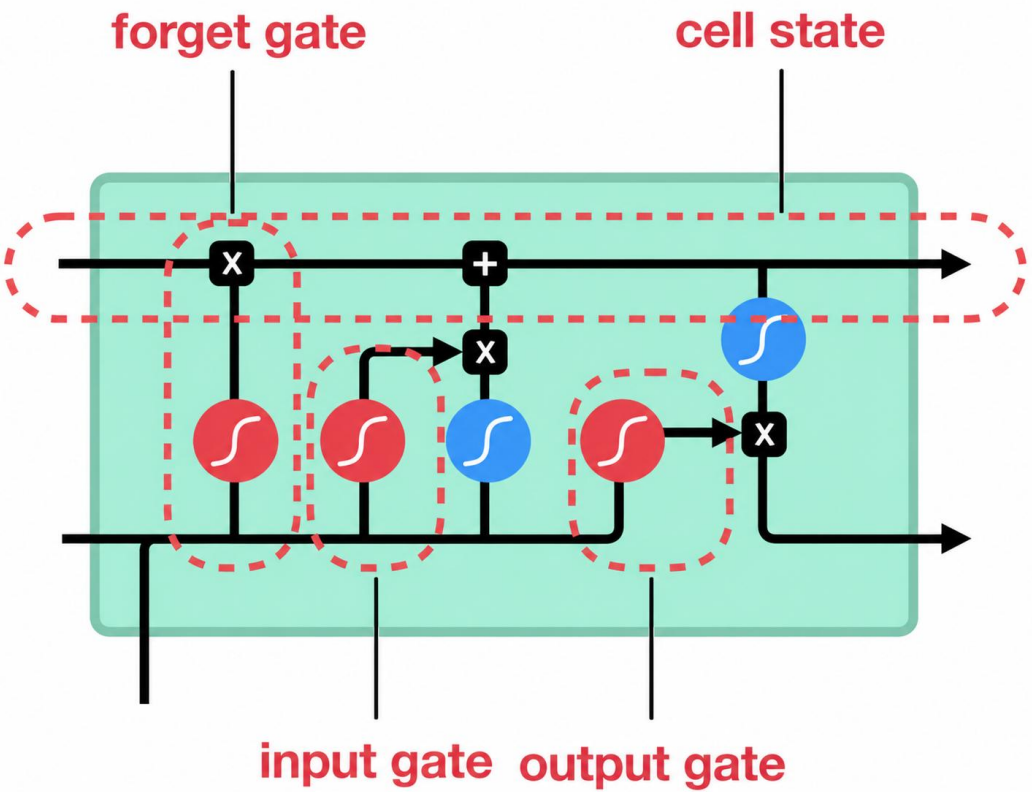
Additional Background and Resources

Sequential nature of Recurrent Neural Networks (RNNs)



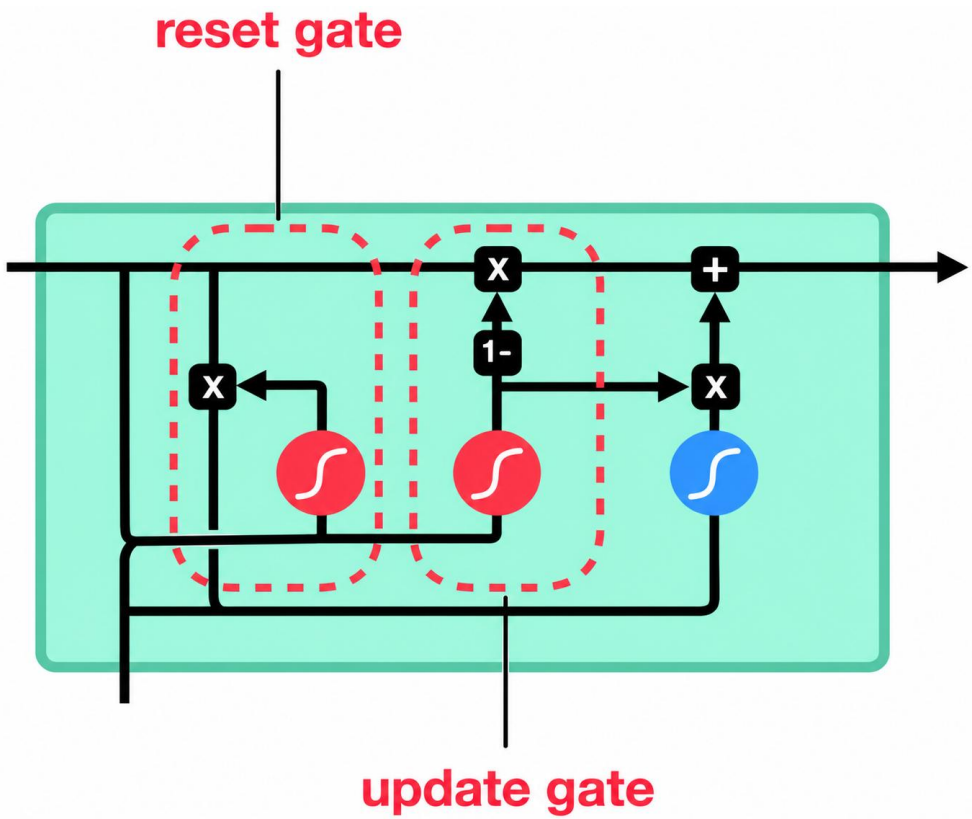
- By **unfolding** the feedback loop in time, we see the actual complexity of these networks. It is as if we train a **very deep network** and that may be very well why they're significantly harder to train.
- With **RNNs** things happen **sequentially** => **deep** graph structure.
- With **Transformers** things happen **in parallel** => **broad** graph structure.
- **Transformers** might simply be **easier to train stably**, and maybe that's why they have better overall results.

Long short-term memory (LSTMs)



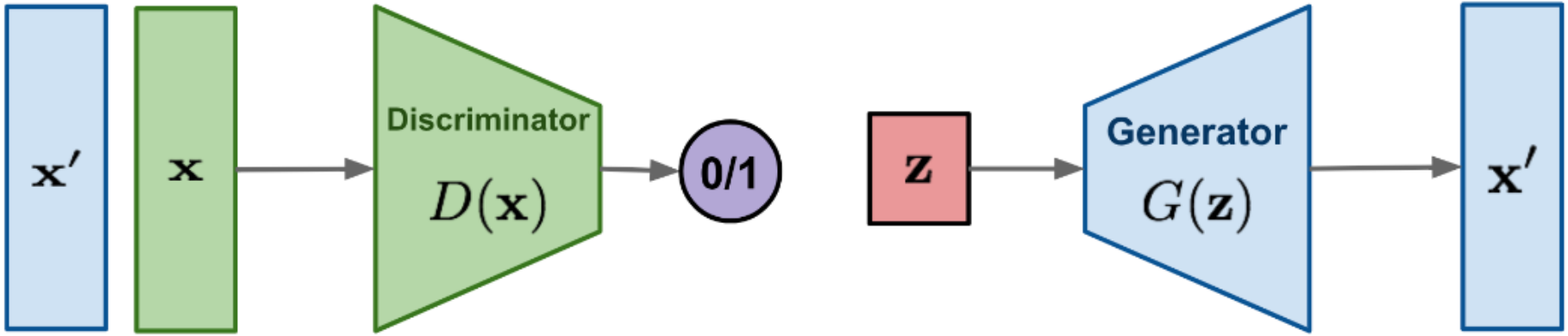
- Contains special **gates** that address the problem of **vanishing gradients**.
- **Exploding gradients** can still occur and are commonly controlled using **gradient clipping**.

Gated Recurrent Units (GRUs)



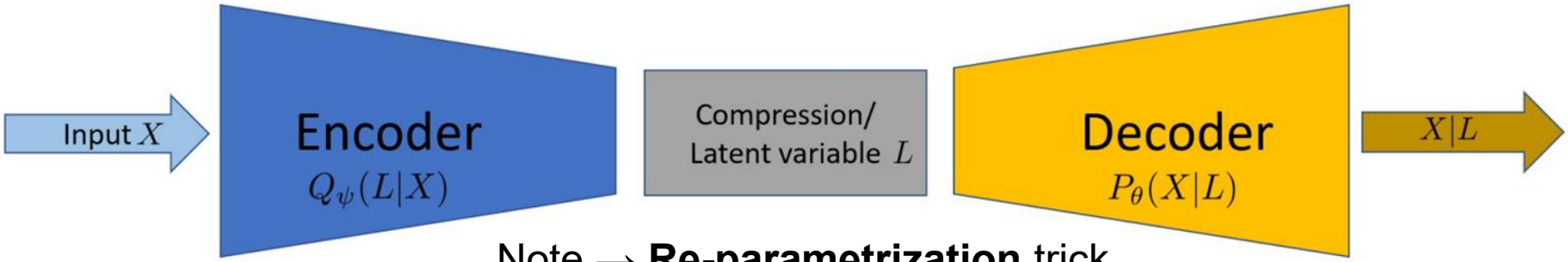
- Generally considered **faster** than LSTMs.
- In practice => **similar outcomes to** what **LSTM** models provide.

Generative Adversarial Networks / GANs



- Two networks: Generator and Discriminator play a **min-max game**
- **Generator** aims at producing realistic outputs **to trick the Discriminator**
- **Discriminator** strives to improve its ability to **discern true from fake** data

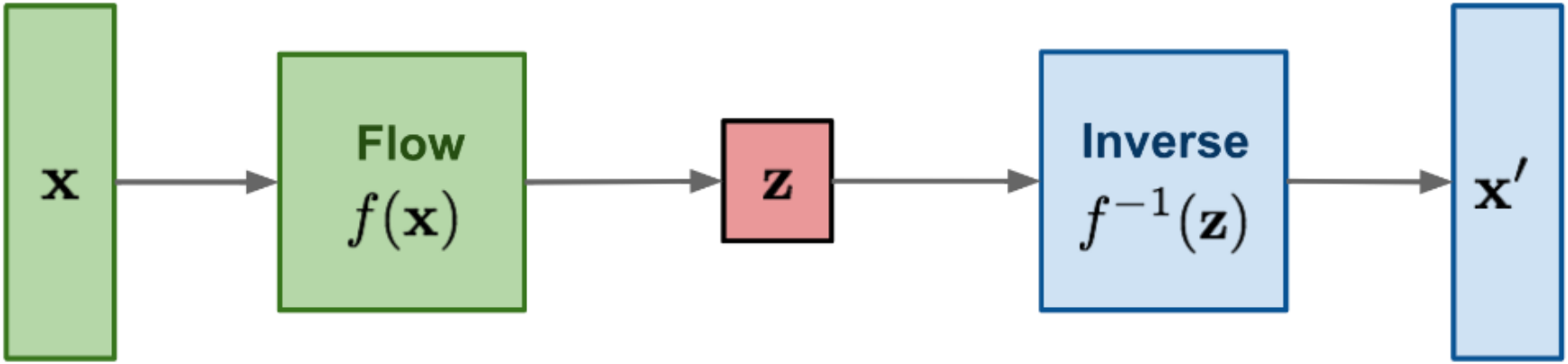
Variational Autoencoders / VAEs



Note → **Re-parametrization** trick

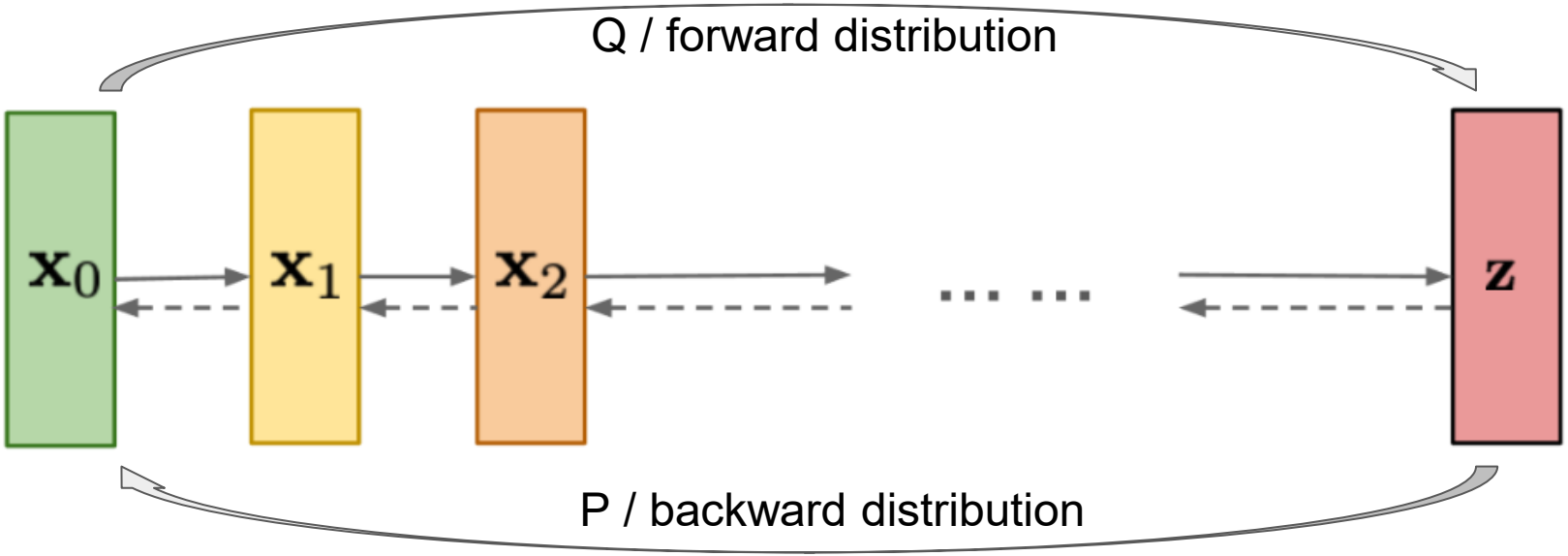
- Inspired from traditional **autoencoders** that **compress** data into **vector codes**
- Variational autoencoders will learn a **latent distribution function** instead
- The **latent distribution can be sampled**, resulting in variable latent samples
- The samples are **decoded**, resulting in a **variety of realistic reconstructions**

Flow-based models



- Use **invertible neural networks** to learn the data distribution
- Enable **exact log-likelihood** directly thanks to **one-to-one** invertibility
- **More** computationally **expensive than VAE**s, due to invertibility requirements

Diffusion models



- **Markov chain** of states, where each state is a noisy version of the other
- The original input X_0 is **diffused into** pure **noise Z** over several steps
- The model **learns the noise that needs to be removed from a state** at time **t**, to get to an **earlier state**

References I

- https://medium.com/data-science/the-differences-between-artificial-and-biological-neural-networks-a8b46db828b7
- https://cs231n.stanford.edu/slides/2019/cs231n_2019_lecture04.pdf
- https://towardsdatascience.com/neural-networks-backpropagation-by-dr-lihi-gur-arie-27be67d8fdce
- https://lilianweng.github.io/posts/2018-10-13-flow-models/
- https://lilianweng.github.io/posts/2021-07-11-diffusion-models/
- https://ducspe.github.io/masterthesis_danucaus/ -> page 18
- “Attention is all you need” by Vaswani et al. -> https://arxiv.org/pdf/1706.03762.pdf
- LLM visualization tool -> https://bbycroft.net/llm
- 3Blue1Brown channel / playlist -> https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- “Swin Transformer V2: Scaling Up Capacity and Resolution” paper by Liu et al. -> https://arxiv.org/abs/2111.09883
- https://www.nature.com/news/2005/050620/full/news050620-7.html
- Grandmother cells paper -> https://www.nature.com/articles/nature03687
- https://www.researchgate.net/publication/7770938_Invariant_Visual_Representation_by_Single_Neurons_in_the_Human_Brain
- https://deepmind.google/blog/alphazero-shedding-new-light-on-chess-shogi-and-go/
- https://www.fz-juelich.de/en/news/archive/press-release/2022/first-european-exascale-supercomputer-coming-to-julich
- https://odsc.medium.com/reinforcement-learning-with-ppo-6a46e79a8359
- “An image is worth 16x16 words: Transformers for image recognition at scale” by A. Dosovitskiy et al. -> https://arxiv.org/pdf/2010.11929v2.pdf
- https://medium.com/@RaghavPrabhu/understanding-of-convolutional-neural-network-cnn-deep-learning-99760835f148
- https://cs231n.github.io/convolutional-networks
- “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows” by Liu et al. -> https://arxiv.org/pdf/2103.14030.pdf
- https://en.wikipedia.org/wiki/Short-time_Fourier_transform#/media/File:Spectrogram-19thC.png
- “Transformers in Time Series: A Survey” paper, by Wen et al. -> https://arxiv.org/pdf/2202.07125.pdf
- https://github.com/qingsongedu/time-series-transformers-review
- AlphaFold2 paper -> https://www.nature.com/articles/s41586-021-03819-2
- https://medium.com/data-science/recurrent-neural-networks-rnns-3f06d7653a85
- https://medium.com/data-science/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21

References II

- https://ducspe.github.io/DCPresentations/#diffusion
- https://www.youtube.com/watch?v=LytU887jCvU&t=305s
- https://ducspe.github.io/DCPresentations/#tokenization
- https://www.fz-juelich.de/en/ias/jsc/systems/supercomputers/apply-for-computing-time
- https://lmstudio.ai/
- https://openai.com/index/navier-stokes-solution/
- https://deepmind.google/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-mathematical-olympiad/
- https://huggingface.co/docs/transformers/model_doc/time_series_transformer
- https://ducspe.github.io/DCPresentations/#alphafold2
- https://nazimbouatta.scholars.harvard.edu/lectures
- https://openaccess.thecvf.com/content/ICCV2023/html/Peebles_Scalable_Diffusion_Models_with_Transformers_ICCV_2023_paper.html
- https://poloclub.github.io/transformer-explainer/